Ville NEA (3rd submission)

23 January 2023 21:10

GCE Computer Science (7517)

The Practical Project

Centre number	Centre name
22133	The Perse School

Candidate's full name

Ville Taipale

Candidate number

Project title

Battleships

Contents

1 Analysis *(9 marks)* 3
1.1 Identification of the problem 3
1.2 Research carried out 4
1.3 Identification of the prospective user(s) 11
1.4 Numbered measurable, appropriate specific objectives of the project 12
1.5 Modelling diagrams 15
2 Design *(12 marks)* 18
2.1 System design overview 19
2.2 Algorithms 25
2.3 Data structures 25
2.4 File structure and organisation 36
2.5 Database design 37
2.6 SQL queries 37
2.7 User interface design (HCI) 39
2.8 Hardware selection/design ..... **Error! Bookmark not defined.**
3 Technical Solution *(42 marks)* 41
3.1 Code listing 52
4 System testing *(8 marks)* 98
4.1 Test plan 98
4.2 Annotated screenshots of test results ..... **Error! Bookmark not defined.**
5 Evaluation *(4 marks)* 103
5.1 Comparison of project performance against the objectives 103
5.2 Effectiveness of the solution 103
5.3 Analysis of user feedback 104
5.4 Possible improvements 105
6 Appendix 106
6.1 User feedback (authenticated by assessor) ..... **Error! Bookmark not defined.**

1 Analysis *(9 marks)*

Analysis		
Level	**Criteria**	**Mark**
3	Fully or nearly fully scoped analysis of a real problem, presented in a way that a third party can understand. Requirements fully documented in a set of measurable and appropriate specific objectives, covering all required functionality of the solution or areas of investigation. Requirements arrived at by considering, through dialogue, the needs of the intended users of the system, or recipients of the outcomes for investigative projects. Problem sufficiently well modelled to be of use in subsequent stages.	7-9

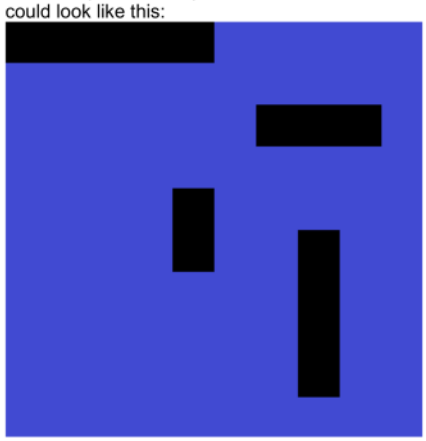
In the analysis section we are looking for:

- A detailed description of the problem / investigation
- Clear evidence that research has been performed (dialogue)
- A clear set of objectives that will be useful across the later sections of the project
- Modelling of the proposed solution that will be of use to later design work

? Having read the analysis stage:
Do you understand what the project is going to do?
Has the student set clear objectives that are detailed enough?

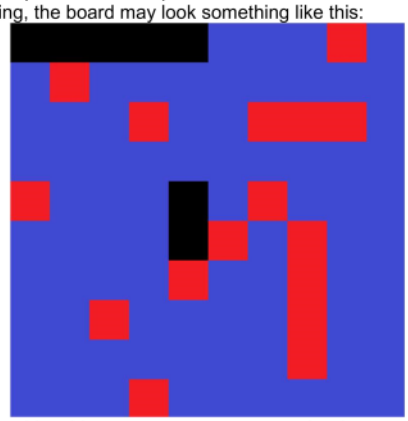
1.1 Background to/ Identification of the problem

Battleships is an imperfect info strategy and guessing game where two players attempt to sink each other's hidden ships. The game starts with both players being given two 10x10 grids. On one of these grids the players should place their ships. Ships generally have one dimension and take up two to five squares on the grid. Ships cannot be placed on top of each other.
An example layout of ships could look like this:



Where the black pixels are ships, and blue pixels are empty squares.
Then the game begins and turns alternate. For the attacker, a turn consists of choosing a square to shoot. If the other player has a battleship on that square, the player should be told that the shot was successful. The places that a player has shot should be marked on the second blank 10x10 grid they were given. Each marking should indicate whether a ship was hit or not, often by colour coding the

markers. When being attacked, a defender should see where the attacker hit on the grid with their ships. The game continues until all squares with a ship have been hit.
After a few rounds of attacking, the board may look something like this:



Where the black squares are ships, blue squares are empty, and red squares are where the player has attacked. Both players should have two grids, one showing their own ships and where the enemy has attacked, and another showing where they have attacked on the enemy's board.

For a complex game such as battleships, there exist a variety of alternative rules. In one game mode, there are different kinds of attacks that can be used. The major ones which I have found and have been mentioned by my clients are a 'barrage' attack which hits more than one square at once (often a 3x3 area) and a 'radar' which does not hit anything but covers a wider area and tells the attacker if any ships were detected in that area or tells the player where on the enemy board there is a ship. I have also found a 'mine' which is placed down together with your ships, and if the enemy hits the square with the mine on it, an area around the mine on the grid of the attacker is destroyed. In addition to this, there can be different kinds of battleships and grids and modes with more player. The battleships are often one dimensional of size 2-5 squares and cannot be customised. Grids often have the same size and rule set. The rule set can be changed to, for example, not tell the attacker if they have hit a ship or not keep track of where the player has hit in earlier turns.

However, the biggest problem currently is that most of the battleship's games which have been iterated on and developed fully are only available physically, meaning that players must be next to each other to play, and it is very difficult to play on the go. In addition, if a game is interrupted, saving its progress can be cumbersome. My clients would prefer something more dynamic and easier to use. It is especially important that everything in the can be customised within the game to their liking, and to make sure that the game does not become boring.

Is this a detailed description of the background/ identification? Is there evidence of an understanding of how the current system contributes to the problem? Is there evidence that the user is well understood and the context is understood? If a game or puzzle, have you described the rules in sufficient detail along with any alternative rule options that may exist? Is there a problem related to having to carry a physical game around or not having time to play a full game in one sitting and the associated issues with packing away and restarting? If a puzzle is there an issue with getting access to enough different puzzles to become proficient? Is there any way to help become better at the game or puzzle?

1.2 Research carried out

1.2.1 Other implementations of Battleships

In order to gather a better understanding about the kinds of things which my program requires, and also what is currently lacking in other programs I searched for digital Battleships games made by other people.
The most complex web application of battleships that I was able to find was battleshiponline.org.
Upon starting the game, a menu with four buttons shows up.



All screenshots are from battleshipsonline.org

Pressing the “i” button creates another menu with the name of the developers of the game:

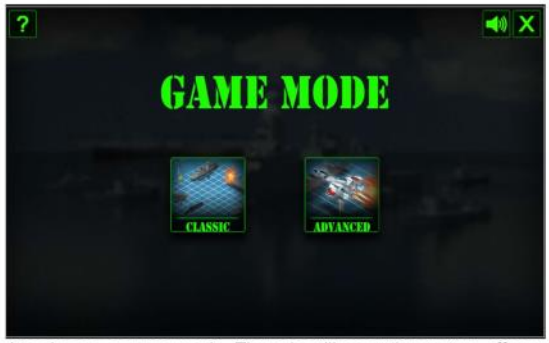


Clicking on the name leads the user to codethislab.com which is the developer’s website and provides information about them and their other games. The user can also press the ‘X’ to close this menu.

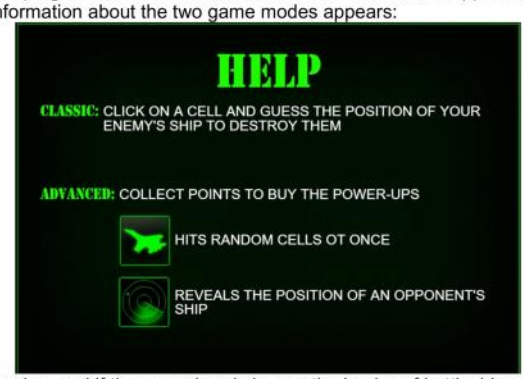
Pressing the  button makes the game take up the user’s entire screen. It can be pressed again to go back.

Pressing the sound button toggles the sound of the game.

Pressing the play button leads the user to another menu:



The user is prompted to choose a game mode. There is still an option to turn off sound and return to the original screen in the top right of the window. An additional "?" button also appears. Upon pressing it, another menu with information about the two game modes appears:



The information is concise, and if the user already knows the basics of battleships, it is enough. However, it does not explain the basics of the game. To exit this menu, the user must click outside of it.

Once the player has returned to the 'Game Mode' menu, if they press 'CLASSIC', a new menu appears prompting the user to place down their ships:



The user can click and drag the ships onto the grid. They can also press the shuffle button, placing the ships randomly. They can also press the trash button to remove all ships from the grid. There is still an option to toggle the sound and return to the original menu by pressing the buttons in the top right of the

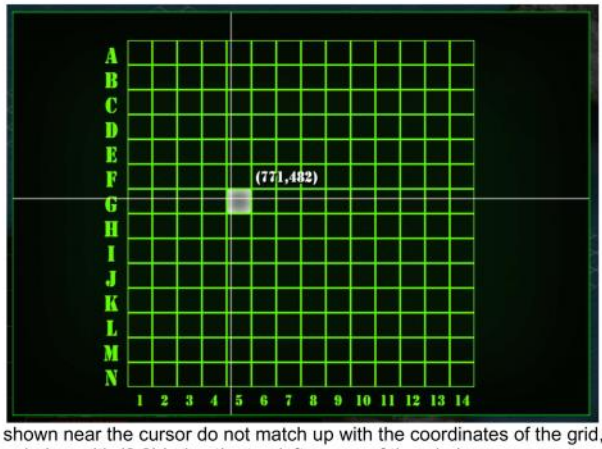
menu. There is no option to change the ship shapes or the number of ships. There is also no option to change the size of the grid.
When the user begins to place ships, another button appears which allows them to rotate the most recently clicked ship 90 degrees:



The application does not allow ships to be placed directly next to each other, as shown by the green areas which appear when placing ships.
When all ships have been placed, a final play button appears which starts the game if pressed:



When the game begins, the user is shown a 14x14 grid and can click on one of the squares to attack it.

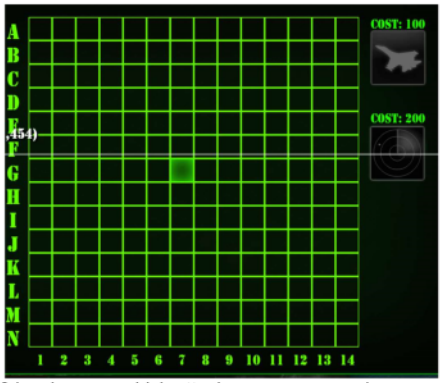


The coordinates shown near the cursor do not match up with the coordinates of the grid, but rather of the entire application window with (0,0) being the top left corner of the window.
After clicking, an animation showing the users missile and AI's missile is run:



The screen also shows information about the user's and the AI's battleships/fleet. The user's 'score' is also shown. For every hit the user lands, they are given 30 points, and for every hit the AI lands, they lose 5. The game continues until one side has no remaining ships. When the game ends, the user returns to the original menu and may repeat any actions they could previously.

When at the 'Game Modes' menu, if the user clicks on the Advanced option, the same ship appears process as before appears. The game is very similar to the 'classic' mode, however in this mode, the user may use their score to buy one of two different attacks before attacking:



The attack which costs 100 is a barrage which attacks numerous random squares simultaneously. The attack which costs 200, is a radar which reveals the position of an enemy ship.

Conclusions:

While the game is fully functional with a GUI and two different game modes, it lacks a lot of the things which my client needs. When asked what she wanted the game to look like, Elvira (my client) said: "I don't really mind as long as I have the option to customise it myself.", and "as long as I have the choice, I'm happy with it." The user is not able to customise anything in this game except making the screen full screen or not full screen. In addition, Elvira said she would like "at least three, easy medium and hard" AI difficulties and that she enjoys playing against "friends in a multiplayer environment". To implement the different difficulties, I will use a simulation which is given the current state of the game (as a two-dimensional array) and returns the most likely position of an enemy ship. The hardest AI should attack that square, and the easier once should have a slightly higher change of attacking that square and other likely squares. She would also like individual accounts which keep track of the "statistics from the games between people and against AI". These statistics would include "the fastest times [she's] had against people or AI" and "hit accuracy and amount of wins". Additionally, when she plays against her friends, they are not always evenly matched, she would like for the game to be 'easier for one side to make the game fair, maybe even looking at our past wins to see if we need help against a certain opponent'. An algorithm which allows the user to change the rules of the game needs to be implemented. Specifically, the user should be able to change the grid size, number of ships, special weapons, and hints for both players.

1.2.2 Research into required algorithms

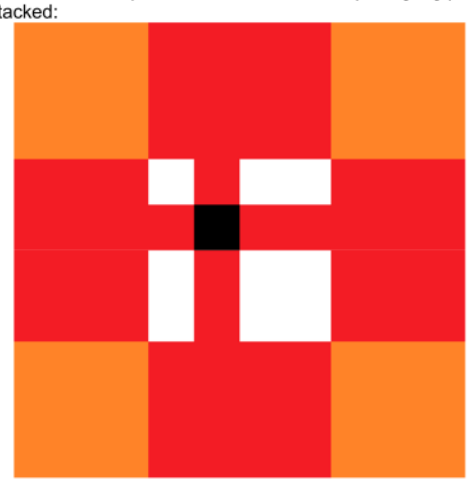
Algorithm for AI gameplay

To implement an AI opponent into my game, I will have to consider the two stages of battleships, placing down ships, and attacking.

For ship placing, I think that randomly placing down ships will be the best approach. While in general, humans will have certain patterns of where they tend to attack, these patterns are often difficult to predict. In addition, if the AI placed down their ships using a specific strategy (e.g. not near the centre), this could quickly become predictable, and a main worry my client had was getting bored of the AI. Placing down ships randomly would prevent the AI from becoming predictable and should not place the AI at a significant deficit over multiple games.

For the attack phase, because Battleships is an imperfect information game, the AI will have to consider the probability of a ship being contained within each square on the board. Then the AI should attack the square with the highest probability of containing a ship. The algorithm which computes probabilities will need to consider the current board state (i.e. what squares have already been hit) and what ships are left. There are multiple ways to compute these probabilities. The AI can:

1. Consider every possible layout of ships, then look through these layouts and see where the most ships were placed.
2. Randomly place down ships on the grid thousands of times, then see where the most ships were placed.
3. For each square in the grid, the AI can consider how many directions a ship can be placed in. For example, in the square [0,0], you can either place your ship facing right or facing down, but not facing up or left. After this, the square which allows for the most possible directions should be attacked.
   A heat map showing the number of possible directions if the square [4,4] (indexing starts at 0) has already been attacked:



   White = 4
   Red = 3
   Orange = 2
   Black = 0 (already attacked in this case)

All these algorithms will effectively create a heat map by forming a two-dimensional array with each square representing the squares probability of containing a ship. The algorithms will then return the index of the square with the greatest probability of containing a ship.

To consider which of these algorithms I should implemente, we should think about their major differences. For example, algorithm 2 is the only algorithm which uses randomness, therefore meaning that the AI attacks would be slightly more unpredictable, further helping with my clients worry of getting bored.
However, an issue which algorithm 2 shares with algorithm 1 is the computation time. For algorithm 1, there will most likely be millions of permutations, and the number of permutations will increase exceedingly fast as the width and height of the game board increases. Algorithm 2 will also have the same issue since to guarantee attacking the best square, you will need to simulate placing ships very many times. However, reducing the number of simulations could make the AI attacks more interesting and increase the game's replay value.
Because of these problems I will most likely implement algorithm 3 for my game as it will likely run much faster than the other two. However, I should also consider algorithm 2 due to the random element.

Hashing algorithm
Because users will be able to create accounts, I will need to be able to store their details securely. For this I will require a hashing algorithm to create a hash of the user's password. The general idea of a hashing algorithm is to create a large seemingly random string from a given input. Important things to consider:

- For a given input, the output must always be constant, unique to all other outputs of the hash function. This is because we want to validate user login by sending a hash of the password to a database. If the hash is not unique, we could accidentally allow someone with the wrong password to log in. If the output was not constant for a given input, then the user could not log back into their account every time.
- It should also be very difficult, if not effectively impossible to find the input for a given output. This is an added security measure so that even if someone gains access to the database or is able to intercept communication and can read the password hashes, they would still not be able to log into any accounts since they would still not know the passwords which created those hashes.

It is also important to note that two separate users should not get the same password hash just because they have the same password. This is because if someone gained access to the database, and saw many repeated passwords hashes, they would know that each of those users have the same password. Therefore, by getting one password they could log into all of the accounts. To prevent this, I will hash a combination of the user's password and their username (which has to be unique).

You must include detailed evidence of research into the background of the project including any knowledge acquisition for the problem domain. Include screen shots or diagrams of the **current system** and interview **transcripts** with the current user. Ensure questions are **detailed** and really scope out the full complexity of the new system and show how it can solve the problem. Identify any **key algorithms** that you may need to write – either 'complex user defined' ones to implement the rules of your program or known algorithms e.g. Minimax. Include an analysis of how deep (how many levels) you are likely to be able to go in a minimax implementation based on your product. Does your game allow for **repeat turns** and so would minimax need to be modified from its pure form? Identify any key **data structures** that might be useful in building your solution. Are there any **research papers or web articles** that describe a **playing strategy** in your product that you will be able to implement? Note that you are not implementing the algorithm, but identifying it as 'needing to be implemented'. Make specific links from the research of existing solutions to the project objectives. Will your product use any networking (client/server) and if so what networking technology will you use (websockets?)

1.3 Identification of the prospective user(s)

An interview with my client, Elvira at 11:30 29/03/2022 at The Perse School.

Hello, nice to meet you. So, have you played battleships before?

- Hello, yes, I have but not for a while, so I'd like to have something different to try, with different levels of difficulty that could ease me into the game.

What do you think is currently missing from battleships?

- I'd say maybe different game modes, I sometimes find myself getting bored of the same mode. And maybe also customisability. **[Stage 4, Objectives 13-15]**

When you play battleships, how do you like playing it and with who? Mainly with who?

- Well, I enjoy playing it either against a bot like an AI to practice and improve myself and also against friends in like a multiplayer environment. **[Objective 51]**

You wanted to play with others, do you think online or local or both?

- I think either would be good, I think online would be more -- easier to play **[Stage 9 Network Games, Objectives 88-93]**, cause you could play anywhere, but either would be good.

When you play with your friends do you think that you're always evenly matched? That it's always a fair game?

- Not really, so it would probably be quite good if there could be a way before the game starts, it would be maybe easier for one side to make the game fair, maybe even looking at our past wins to see if we need help against a certain opponent. **[Objectives 85-87]**

Since you'll be playing like locally as well or online, do you think you'll be sharing the program with someone else?

- Yeah.

Do you think you need accounts for the different people you'll share it with.

- Yeah, to keep the statistics from the games between people and against AI that I've won and maybe also lost, also maybe statistics of ongoing games. Also, maybe statistics of the fastest times I've had against people or AI. Then also hit accuracy and amount of wins. That would be probably useful. **[Stage 8 User Accounts, Objectives 69-70, 75-84]**

Do you think you'll need security for these accounts?

- Yes, encrypted passwords would be very useful. And I'd like the login time to be quite fast, since there are some websites and apps where I have to wait quite a long time to get in. **[Objectives 71-74]**

What do you think you'd like the actual game to look like, the board mainly and the ships.

- I don't really mind as long as I have the option to customise the grid and ships myself. I'd also like various resolutions for the GUI. As long as I have the choice, I'm happy with it.
  [Objectives 14, 39-42]

Earlier you mentioned different rule sets. Could you be more specific about what you would like?

- Yeah, I like the modes with different types of attacks like radar and barrage, but I'd also like a classic mode.
  Also, before the game starts, it's nice when it allows the user to set up the basic rules such as how many ships, battlefield size, what kind of ships, AI difficulty, all of that.
  [43-50]

You would like to practice against AI as well?

- Yeah, that would be useful.

So, what levels of difficulty do you think these AI should be?

- I'd say at least 3, easy medium and hard. Maybe the easiest one could have an undo move or some hints or other help **[Objective 14]**. The hardest one should be very good, and I'd like me not to be able to beat it every time even after a while of playing. So, I'd like to be challenged even after I improve.
  [Stage 5 AI Opponent, Objectives 51-56]

In addition to that interview, I also received feedback from my teacher Mr Gray:

Mr Gray:
I really like Battleships, but the size of the board makes it impractical in many cases since you can't really carry it on the train or similar. I find that the game is really popular when I invite my friends around, but only two people can play at once. I wonder if technology would make it possible for more people to play at the same time – maybe four people? **[Stage 6 Multiple Players, Objectives 57-61]** It would also be nice if I could play against a computer so that I can practise. Sometimes when my friends and I play we find the gameplay quite slow (only firing one shot at a time) so we wondered if there could be different 'weapons' that are somehow bought with 'credits' in the game – e.g. a weapon that fires a 3x3 shot to cover larger space? Really just something to make the game more modern and speed up the gameplay. A computer opponent would be very helpful too, although my friends have many different skill levels.

This is a short paragraph that identifies the user and describes them and their need. It should include detailed interviews with the prospective user (not superficial yes/no questions) where the user's opinion is sought in detail. Make specific links from the user's requirements to the objectives of the project.

1.4 Numbered measurable, appropriate specific objectives of the project

The first three stages are objectives which set out my plan for how to implement Battleships. The stages after that describe objectives which help me implement the more specific requests made by my clients (for example, adding special modes, user accounts etc).

Stage 0 Basic Terminal (MVP):

1. The program should be runnable with a ‘t’ option to run in the terminal.
2. At the start of the game, a two-dimensional array should be displayed and four grid references to place down a ship in the format ‘x,y,v’/’x,y,h’ should be accepted. The x gives the left most square of a ship, and the y gives the topmost square of a ship, v/h are for vertical/horizontal. This needs to be repeated for both players.
3. The locations of ships should be stored in a two-dimensional array.
4. At the start of a turn, the players previous attacks and current ships will be printed in the terminal.
5. One grid reference indicating where to attack in the format ‘x,y’ will be accepted from the user.
6. After each turn input will pass to the other player and information about their current ships and previous attacks should be presented.
7. When one side has no squares with an intact ship, the game should end and declare the winner.

Stage 1 Full Terminal:

8. Running the program without a ‘t’ option should display an error.
9. When placing ships, the user should not be allowed to place them so that ships overlap or do not fit in the grid.
10. When attacking, the user should not be allowed to attack a square they have already attacked.
11. When attacking, the user should not be allowed to attack a square outside of the grid.
12. After a turn is over, the terminal should be cleared, preventing the second user from seeing information about the first users ships and attacks.
13. There should be more than one game mode.
14. Before the game starts, the user should be able to decide the grid size, number of ships, ship types, special attacks and hints.
15. A game mode which allows more than two players to play against each other.

Stage 2 basic GUI:

16. Running the program with the ‘g’ option should run the GUI version of the game, displaying “Help”, “Play”, and “Quit” options.
17. These options should be selectable by moving and clicking with a mouse.
18. Upon selecting the “Quit” option, the GUI should close, and the program should halt.
19. Selecting the “Play” button should display a new GUI showing different game mode options.
20. Pressing the escape key or exit option should close this new GUI, returning to the main menu.
21. In the Play window, choosing the classic mode option should start a new game, displaying a GUI for placing down ships.
22. The user should be able to drag and drop ships from a list onto a 10x10 grid.
23. Upon pressing the ‘R’ key, the most recently selected ship should rotate 90 degrees.
24. The user should be able to press a “Ready” button, allowing the second user to place down their ships.
25. The second user should be given the same blank grid and ships the first user was given.
26. The locations of both users’ ships should be stored in two different two-dimensional arrays.
27. Once the second user has pressed the “Ready” button, a new GUI prompting the first user to attack should be shown.
28. The first user should be able to select a square on the grid to attack.
29. While thinking of where to attack, the user should be shown two grids. One with their previous attacks, and another with all of their current ships and their state.
30. Upon attacking, the other user should be prompted to attack.
31. After each attack, the program should check if the attack hit, and updating the two-dimensional array containing the other user’s ship’s if it hit.
32. This cycle should continue until all ships in one two-dimensional grid have been fully destroyed.
33. The loser is the one who’s two-dimensional grid containing ships is first destroyed.
34. The winner should be displayed, and the GUI should return to the main menu once someone has won.

Stage 3 Full GUI:

35. The user should be able to change the resolution of the window.
36. In menus, things should be selectable using either the mouse or keyboard presses.

37. When turns change, the program should change to a blank screen, then after any input from the user change to the other users board.
38. Pressing the “Help” button should open a menu displaying information about battleships and how this version of battleships works.

Stage 4 Full Game:

39. An additional “Options” button should be added to the main menu.
40. Upon pressing the options button, an options menu should appear, showing the different things the user can change.
41. The user should be able to customise the resolution.
42. The user should be able to customise what the GUI looks like. Mainly the colour scheme.
43. An additional game with different attacks should be included.
44. The user should be able to select this mode after pressing the “Play” button in the main menu.
45. In this game mode, before it starts, the user should be able to choose which attacks should be included in the game.
46. These attacks include barrage, mine, and radar.
47. The user should also be able to toggle a “Hit Streak” button which allows the player to attack again if they hit a ship in the special mode.
48. Barrage attacks multiple squares.
49. Mines should be placed down by the user while they place their own ships, and if an enemy hits a mine during gameplay, a ship on the enemy’s square takes one damage.
50. Radar finds the location of an enemy ship.

Stage 5 AI opponent:

51. Upon selecting a game mode, the user should be able to select to either play against AI or another player locally.
52. The user should be able to choose between three different difficulty levels of AI.
53. The harder AI should always attack the most likely square where an enemy ship may be.
54. The easier AI should attack more randomly, with a weighted probability of attacking better squares.
55. Upon hitting a ship, all AI should hit clockwise around the hit ship to find where it continues, and then shoot in that direction until the ship ends.
56. Once the AI misses in that direction and the ship is not yet destroyed, the AI should attack the opposite side of the original hit.

Stage 6 Multiple Players:

57. An additional option allowing for upto 4 players to play against each other.
58. During each turn, a player attacks all other players.
59. A player is knocked out when their ships have been destroyed.
60. They should no longer have a turn, and no one should be able to attack them anymore.
61. The winner is the last remaining player.

Stage 7 Save Games:

62. In the middle of a game, the user should be able to press a button to save and quit the game.
63. Upon saving a game, the state of the game should be uploaded to a central database stored on a server.
64. In the main menu, an additional “Load Game” button should be added.
65. This menu should display information about save games and/or the ID which allows the user to load them.
66. If the user selects a save game, the user should be sent to that game to play against the correct opponent.
67. The save game must contain the locations of both sets of ships, the type of opponent, and which player owns the respective states.
68. The user should be able to leave the Load Game menu by pressing the Escape key.

Stage 8 User Accounts:

69. When first opening the game, the user should be given the option to play as guest, log in or sign-up.

70. The user should be able to play the game as a guest or with an account.

71. Logging in should allow the user to input a username and password.
72. The stored password should be encrypted.
73. If the username and password match, the user should be let into their account.
74. Look-up time for passwords should be quick.

75. Pressing the sign-up option should allow the user to create a new account with a new username and password.
76. The user should only be allowed to make a new account if the username is unique.
77. After registering the user should immediately be logged into that account.

78. Account information should be stored in a database on a server.
79. An account should keep track of various statistics of the player and the matches they have played.
80. Account information should include number of wins against certain opponents, best time against AI and best number of ships left against AI and overall hit accuracy.
81. When logged in, a user should be able to store saves in a database on a server.
82. When logging in from different devices, the user should be able to see all of the stored statistics and save states.
83. In the main menu, a new option called “Statistics” should exist, showing all of the information about this account.
84. The user should be able to return back to the main menu by pressing the escape key or quit button while in the settings menu.
85. When playing an online game, if both players are logged in, the game should check if one player seems far better than the other.
86. This can be done by checking the amount of wins against each other, and the total amount of games played, hit accuracy wins against AI etc.
87. If this is the case, the game should suggest that the worse player is given an advantage.

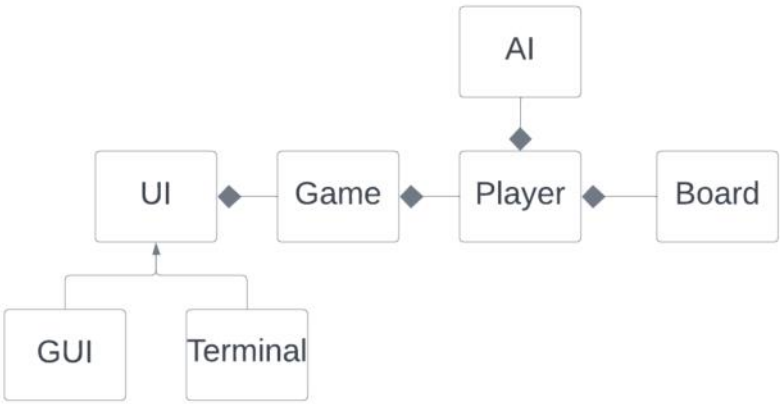
Stage 9 Network games:

88. Two players should be able to play against each other on different devices by connecting to a server
89. To start a network game, both players must be logged in and input the username of their opponent
90. Once a game has started, players should be able to place ships simultaneously.
91. During attacking, players should be able to attack simultaneously.
92. The difference between the number of attacks performed by players should be at maximum one.
93. Everything which can be done in a local game, should be possible in an online game.

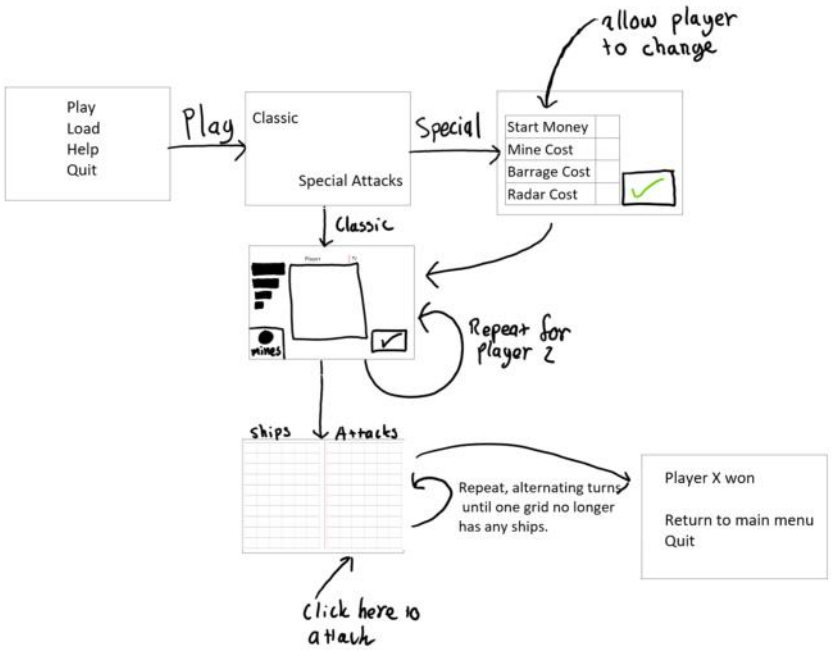
These should cover all required functionality of the solution or areas of investigation ('appropriate' means the specific objectives are single purpose and at a level of detail that is without ambiguity)
NB These must be complex even if you are unable to code them all fully! They should be described in sufficient complexity as to meet the standard required of A level. Each user need should be identified as a measurable objective and then within that objective specific outcomes should be identified and listed. These should not include any add-ons of standard algorithms nor any code that will be generated by a GUI. This is the most important section for you to get right!!!!!!!!

1.5 Modelling diagrams

UML diagram:



Simple GUI:



E.g. E-R model, High level software object model, Data Flow Diagrams, mathematical models, the flow through the program.

Important read – Examiners Report

Analysis There is encouragement that a student should gather details for the project from users via a dialogue of some form. Some of the interviews seen were very detailed and clearly gained relevant information for development of the project. Unfortunately, it was also common to see very short interviews which gathered no real requirements for the project to be assessed highly by centres. Students should be encouraged that in the interview it would be beneficial to ask probing questions to find out the real requirements of the user(s) and not just the kind of colours to be used or whether they like playing games. The analysis should contain a list of the objectives set by the student for their technical solution. It was pleasing to see many students provide a detailed list of objectives that indicated both the requirements to be met and the complexity that this might involve. Students who submitted vague and brief objectives would struggle to pick up high marks in the analysis section and it would also be common for the rest of the project to suffer slightly. Weak objectives also make awarding the completeness mark hard as consideration must also be placed into what an A-level student would be expected to achieve. The analysis section is to contain some modelling of the proposed system and it was pleasing to see students complete this in a variety of ways. Those projects that needed data processing usually included some discussion of the data required and DFD or ER diagrams. Students looking to produce a game sometimes struggled with the modelling section and also left the reader not understanding what their idea actually was. Students completing gaming projects could consider sketching out some ideas for the game and discussing the game flow as part of their modelling section.

2 Design (12 marks)

Documented design		
Level	**Criteria**	**Mark**
4	Fully or nearly fully articulated design for a real problem, that describes how all or almost all of the key aspects of the solution/investigation are to be structured/are structured.	10-12

In the documented design section we are looking for:

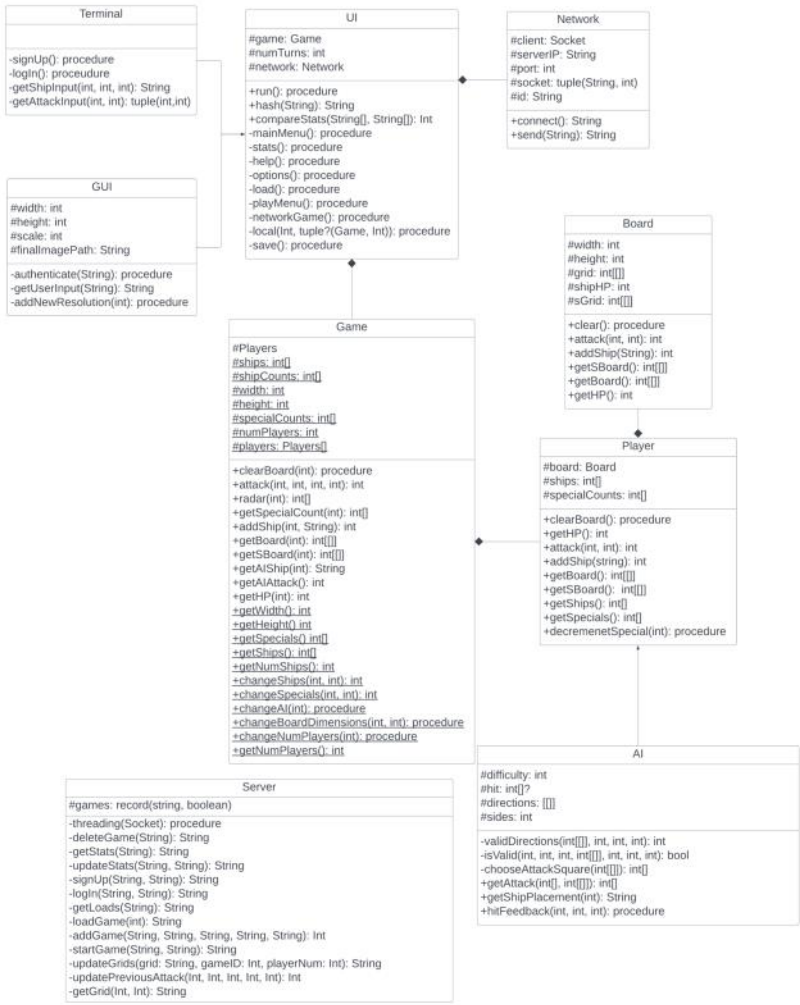
- An overview to the whole system design (module breakdown / objects / units / web pages …)
- Clear design & detail for some of the complex algorithms
- Detail for the data to be used (database design / data structure design)

Having read the documented design:
Do you understand how the project is going to work as a whole?
Do you have a clear understanding as to how some of the complex algorithms will work?
Do you understand how data will be processed / structured?

2.1 System design overview

2.11 Class diagram

A flowchart for the UML design of my code is shown below:



Class explanations

UI

The purpose of this class is to act as a base structure for the GUI and Terminal classes, as well as removing repetitive code from the respective classes.

It contains:

- run() – the procedure that is called when the user starts the game through a terminal.
- hash() – takes a password as input and returns a hash of it to be sent to the central database.
- compareStats() – Takes two strings as input and returns the player that needs help, or -1 if neither player needs help.
- mainMenu() – Displays the main menu.
- stats() – Displays the players statistics
- help() – Displays useful information about the game.
- load() – takes a gameID as input, queries the database with that gameID to get all the required details about the saved game, then sets up that game.
- playMenu() – Displays the menu allowing the user to choose the type of game they want to play.
- getShipInput() – checks if the player is an AI, if it is calls the getShipPlacement() function of the AI class, otherwise it asks for user input, then it returns the result of either of those operations.
- getAttackInput() – same as getShipInput(), except for getting where the player wants to attack next.
- options() – takes user input to update static variables of the game class
- save() – Saves the current game onto the database.

GUI

The GUI is a type of UI. Instead of adding a new menu on top of the old one, I remove the old one, then initialise a new menu to help make my code clearer and make it easier to jump between menus. A GUI has a width, height, and scale. The scale is relative to the scale that images are drawn in (screen = 48x27). If the user wants to play in 1920x1080, then the scale would be 1920/48 = 40.

- authenticate() – Takes the authentication type as input. Instead of having separate logIn() and signUp() methods, I combined them for the GUI class because a lot of code is required to set up the window. It allows the user to login or sign up.
- getUserInput() – Takes a title as input, displays that title, and allows the user to write onto the screen. Returns what the user wrote.
- addNewResolution() – Takes a scale as input and checks if images in that scale exist. If they do, then the scale is update, if they don't the new images are made, then the scale is updated.

Terminal

It is also a type of UI. The terminal implementation is quite easy, so it does not require anything other than what is already implemented in the UI definition. I wanted to have three classes (UI, GUI and Terminal) to show that the GUI and Terminal are separate versions of the same base idea, and also prevent some repetition in my code.

- signUp() – takes a username and password, then if the username is unique, stores it and a hash of the password in the database.
- logIn() – authenticates user login by checking that the username and password hashes match.
- getShipInput() – Takes the player number, ai difficulty and length of ship as input. Returns the location of the square the player wants to place their ship on.
- getAttackInput() – Takes the playernumber and ai difficulty as input. Returns the location of the square the player wants to attack on the enemy's grid.

Network

The network class allows a client to access a server which in turn can access the database. This is useful for user accounts and online games. The class contains:

- connect() – Initiates the connection between the client and server.
- send() – Takes a string of data and sends an encrypted version of it to the server. It returns the response it gets from the server.

Server

The server class is not directly related to any other class, but it does communicate with the network class. Its main purpose is to update the database and connect players together in a multiplayer game. It contains:

- threading() – Called when a new device is connected to the server. Here, string messages are accepted from the network class. Depending on the type of message, this procedure will call one of the other functions.
- signUp() – Takes a username and a password hash. Checks if the username exists in the database, if it does not, then a new row is added to the player table. Otherwise “invalid” is returned.
- logIn() – Takes a username and a password hash. Checks if the password hash matches the stored password. If it does, “valid” is returned, otherwise “invalid” is returned.
- getLoads() – Takes a username. Returns a comma separated string of the gameIDs which the player with that username has saved.
- addGame() – Adds a row to the Game table. Returns the gameID of added game.
- startGame() – Takes two usernames as input and starts a network game by adding a row to the Game table with those two usernames.
- updateGrids() – Takes a grid, playernumber and gameID. Updates a row in the Game table to change one player’s grid.
- updatePreviousAttack() – Update previous attack column of a row with given gameID.
- getGrid() – Returns the grid of a player.
- getStats() – Takes the playerNumber. Returns the statistics for that player.
- updateStats() – Takes playerNumber. Updates the statistics for that player.
- deleteGame() – Takes a gameID. Deletes the row with that gameID from the Game table.

Game

The game class acts as a middleman between the inner functions of the game itself and the UI. The game. It further abstracts away information about what is happening around it, as the only way it knows the “players” is as things that play game, whereas the UI may know about that players’ statistics, accounts, and names. Since that information is not necessary for playing a game of battleships, I believed that it would be help make my code cleaner to add this class.
It contains:

- attack(), getSpecialCount(), addShip(), getBoard(), getSBoard(), clearBoard(), getHP() – Each of these functions take playerNumber as input, then calls that players respective function. (e.g., attack(0) would call players[0].attack())
- radar() – Takes playerNumber as input and searches through that players board until it finds a ship, then returns the index of that ship in the board.
- getAIShip() – calls the getShipPlacement() method of the AI Player
- getAIAttack() – calls the getAttack() method of the AI player
- It also contains static methods which act as mutators and accessors for the static variables of the Game class. These static variables are “settings” for each game the user plays, and they can change each of the to play a game with different rules.
- It also contains static getters and setters for the static variables of the Game class.

Player

This class also helps abstract the game, making my code easier to write and debug. It acts as a base definition for what a player should be. There is also some additional reformatting and information stored within this class to make alternative versions of the base game easier. It contains:

- attack(), addShip(), getBoard(), getSBoard(), clearBoard(), getHP() each call functions within the board class.
- getShips() returns a list of integers. Each integer represents the size remaining ships for that player. For example, if the player had placed down three ships of length 4, then getShips() would return [4,4,4], even if one of those ships had been damaged a little bit. Ships are only removed from the ship list if they are fully destroyed.
- getSpecials() returns a list of integers. [barrageCount, radarCount, mineCount, hitstreak] describing how many of each special are left. HitStreak is a 0 or a 1 depending on whether it is used in the current game.
- decrementSpecial() decrements a counter within the specialCounts array. It is called when the player has used a special attack.

Board

This is where the most fundamental code of the game is implemented.

- attack() – Takes an x and y coordinate as input, then updates the square within the grid with that index to indicate that it has been attacked. It then returns an integer value which describes whether the attack was successful or not, or if something went wrong.
- addShip() – Takes a string input (xyrh) x and y being the coordinates, r being the rotation, and h being the size of the ship (or the health points). It then updates the grid to show that there is a ship at [x,y] facing in the "r direction" with size h. This is done by making every square that the ship touches a 1 (instead of a 0). The function then returns a -1 if something went wrong, or 1 if the ship was placed successfully.
- getSBoard() – Returns the grid with all of the ship location changed to empty squares
- getBoard() – Returns the grid
- clear() – Reverts the grid to only 0's.
- getHP – returns the shipHP.

A board object stores the contents of the "board" using a two-dimensional array, grid. The width and height of this grid should be passed into the constructor when creating a new board object. The board also has a shipHP value which stores the total remaining ship health points within that board

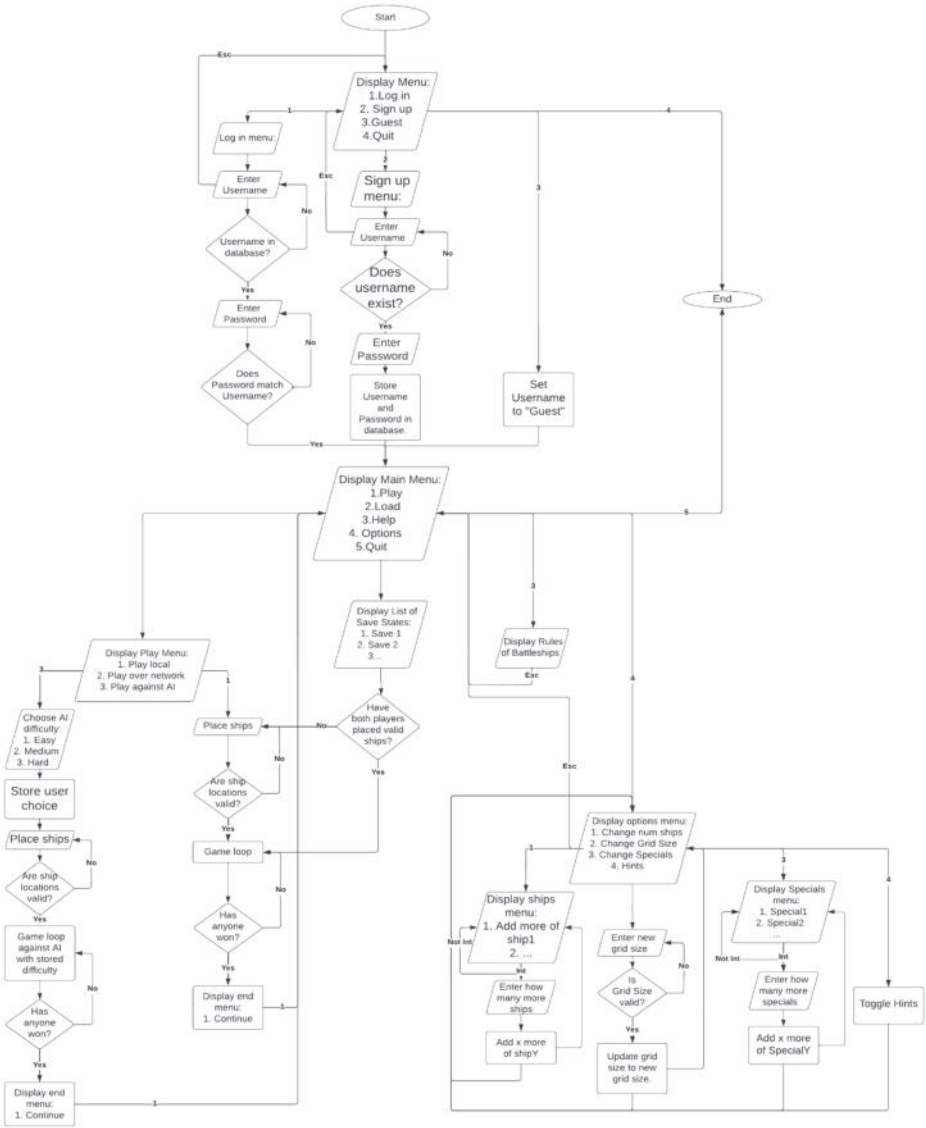
AI

AI inherits from player as it should be able to do anything that a player can do, but also figure out its own moves. An AI object has:

- validDirections() – Takes an x and y coordinate, ship length and board, then returns how the number of directions in which you can place a ship at those x and y coordinates on that board.
- isValid() – Takes parameters of a ship and board, then tells you whether that ship is in a valid position (e.g. does it overlap with other ships, go out of the grid, etc).
- chooseAttackSquare() – Takes a two dimensional array and returns the index of the maximum value in that array.
- getAttack() – Takes a board state and list of remaining ship lengths, then returns the x and y coordinates that are most likely to contain a ship on that board.
- getShipPlacement() – Takes a ship length and returns the index of where it would place down that ship.
- hitFeedback() – The game calls this function after finding out whether the AI's previous attack hit, missed or was valid. The ai then appropriately modifies its "hit" variable. More detail can be found in the algorithms section.

An AI object also has a difficulty, directions array, hit variable and sides counter. The directions array is just a list of up, down, left and right directions in the form [[0,-1], [0,1], [-1, 0], [1,0]] The up and down directions are swapped because the top row has index 0, and bottom row has index (height – 1) when the board is printed. The sides counter and hit variables are used to keep track of whether the AI should be looking for its next ship, or finishing off an already known ship, and in this case how it should destroy the ship.

2.1.2 Full Program Flow Chart



The game starts with the main menu. The user can login, signup, play as guest or quit. If the user wants to signup or login, the database is appropriately queried, if the user wants to play as guest the game immediately moves to the “play menu”. From the play menu the user can start a new game, load a previous game, get help, change the options, or quit. Choosing the “play” options allows the user to choose between a local game, network game, or AI game. Choosing the AI options allows the user to choose the AI’s difficulty, then starts the game. Once the game has started, both players place down their ships, then the attack phase begins.
If the user had originally chosen the options menu, they can change the number of ships/types of ships, grid size, number of special attacks/types of special attacks and turning hints on or off.

2.1.3 Data flow diagrams

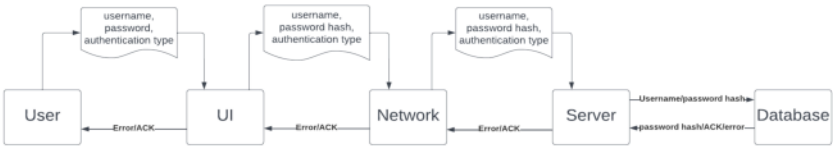
Gameplay phase



This phase is quite simple. When placing ships, the user input is just forwarded by all classes toward the board which places down the ships. When attacking starts, the user passes in the attack square and attack type. This in forwarded to the Game class. The Game class then breaks down the attack type into attacks on the player. For example, a barrage attack would be 9 regular attacks in a 3x3 grid.
At each step user input is validated in some way. The UI knows the type of input it should receive, the game knows about the types of players it has (if one is an AI) and what the attack types are, the Board object knows about the state of the grid and whether a ship or attack location works. The classes use this information to validate user input at each stage and return appropriate error messages or acknowledgements.

Network game
The network game will be similar to the above scenario, except an additional step between the Game and Player classes is required. When placing ships, the game will update a database stored on a server with the locations of the ships. When attacking, the game will update a “recent attack” column in a database. Then the other client’s game class will query the database to receive its attack, then the Player attack function can be called.

User Accounts



When the user wants to sign up or login, they input a username and password. The UI hashes the password and sends it and the username to the network class. The network class sends the username, password, and authentication type to a server.
The server runs on the same device which stores the database. If the authentication type is “login” then the server queries the database to get the password hash with the matching username. If the input hash matches the database hash, then login is successful. If the authentication type is “signup”, then the server checks that the inputted username has not already been used, and if not adds the username and password hash to the database.

Description plus diagrams such as class diagram: For each class describe its purpose (high cohesion) and each public method (interface/low coupling)
Flowchart: Give general top-level flow of the system from running the program. May be more than one flow chart for different use cases.
Data flow diagram: Top level diagram showing how data moves between different parts of your software for different use cases e.g. logging in accesses database, saving access database.

Swim-lane diagrams for networking messages

2.2 Data structures

2.2.1 GUI class

The following data structures are lists which are dynamically defined based on the amount ships the user wants to have in their game. Within the lists, an item with a specific index refers to the same ship across each of these lists. I chose to separate these into their own lists instead of making one large two-dimensional list to make my code easier to read and easier to change if I wanted to add, remove or change some defining characteristic of a ship.

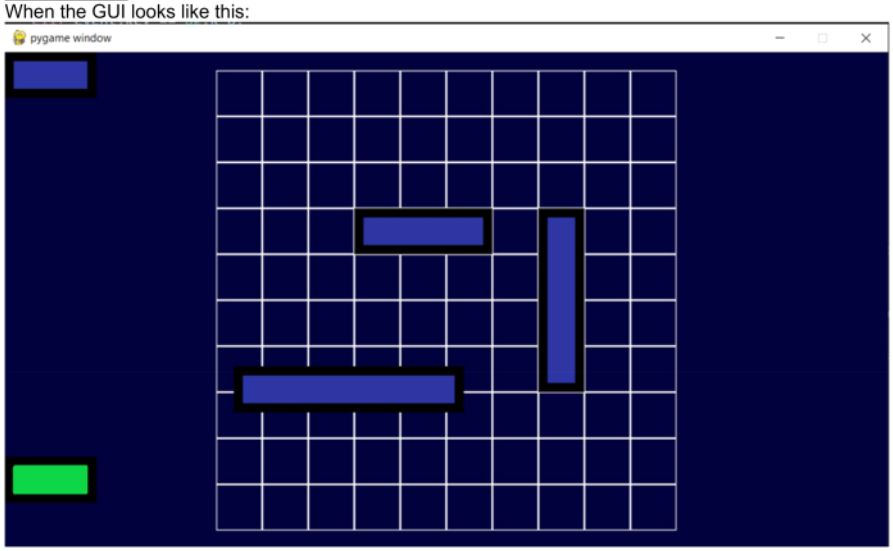
shipAttach → integer list
Each item is an integer which describes the current state of a ship.
0 means the ship has not been placed down.
1 means that ship is currently being dragged around by the user.
2 means that the ship has been placed down.

shipImages → list
A list of pygame image objects for each ship.

shipHp → integer list
A list of the current health points of the ships.

shipRotation → list
The item is a 0 if the ship is horizontal, and a 1 if the ship is vertical.

Example data
When the GUI looks like this:



The previously named data structures will look like this:

```
ShipAttach: [0, 2, 2, 1]
ShipHP: [2, 3, 4, 5]
ShipImages: [<Surface(100x50x24 SW)>, <Surface(150x50x24 SW)>, <Surface(50x200x24 SW)>, <Surface(250x50x24 SW)>]
ShipRotation: [0, 0, 1, 0]
```

In the shipAttach array, the first item is “0” because it has not been moved (top left ship), the second and third items are “2” since those two ships have been placed down (length 3 and 4 ships). The last item is a “1” because that ship is currently being moved around by the user and is following the mouse cursor (length 5 ship).

The shipHP array just tells you the length of the ships.
The shipImages array contains pygame surface objects which represent the images for each of those ships. By calling a pygame “blit” function with these images as parameters, the GUI class makes the images appear on the screen.

2.2.2 Board Class

_grid → two-dimensional integer array
This will be used to store my board. Each item in the grid represents one square on the board. The items can have a value of -2, -1 0 or 1.
-2 means that there is no ship, and the square has been attacked.
-1 means that there is a ship, and the square has been attacked
0 means that there is no ship (an empty square)
1 means that there is a ship

_sGrid → two-dimensional integer array
The secret grid. This will be similar to the _grid array, except all 1’s will be replaced with 0s. This will be the board that the user sees when looking at the enemy’s board.

Example data
The following example data uses 10x10 grids.

Empty boards
Both _grid and _sGrid would look like this:

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

After placing down ships
_grid:

```
[[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 1, 1, 1, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

_sGrid would remain unchanged as a 10x10 grid filled with 0s.

During the attack phase

Grids could look like this.
_grid:

```
[[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [1, -1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, -1, -1, -1, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

_sGrid:

```
[[0, -2, 0, 0, -2, 0, 0, 0, 0, 0],
 [0, -1, -2, 0, 0, 0, 0, 0, 0, 0],
 [0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, -1, 0, -2, 0, 0, 0, 0, 0, 0],
 [0, -1, 0, 0, -2, -2, 0, 0, 0, 0],
 [0, -2, 0, 0, -2, -1, -1, -1, -2, 0],
 [0, -2, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, -2, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, -2, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

2.2.3 AI class

A very important part of the AI attacking is its ability to completely destroy a ship after landing one hit on it.

_directions → constant array of tuples [(0,-1), (1,0), (0,1), (-1,0)]. Each tuple represents a direction. They are in the order [Up, Right, Down, Left].

_hit → integer array in format [x,y,direction,amount] or None type
This keeps track of the original hit [x,y], the index of the direction to attack next relative to [x,y] and the amount of steps in that direction. This helps the AI completely destroy a ship once it has already found one part of the ship. After every attack, the game class calls a hitFeedback() function in the AI to tell it whether the last attack was a hit, miss or if an error occurred. The AI uses this to change the hit array. If the hit array is None, it means that the AI should continue searching for ships. If it is not none, then it means that the AI should attack the square with index:
[x + _directions[direction] * amount, x + _directions[direction] * amount]
For example if the AI had just attacked the square [5,5] and hit a ship, then the _hit array goes to [5,5,0,1] meaning that the AI should attack [5,4] next. If the [5,4] attack is successful, then the _hit array goes to [5,5,0,2] meaning that the AI should attack [5,3] next. If that attack fails, the _hit array goes to [5,5,2,1], meaning that the AI should attack [5,6] next. If that attack fails, the _hit array goes to None, meaning that the AI should go back to searching for ships.

2.2.4 Server class

The server class only has one important data structure.
_games: list of records. record(String, String, Boolean, Boolean): [username1, username2, hasConnected1, hasConnected2]
or list of integers..
This record is used to help keep track of which players have connected to the game. When the first player connects, both the username are added the record, and hasConnected1 is set to True. Once the second player has connected, a gameID is assigned to the game, and the record is changed to that gameID.

Give examples of what data would be stored in the data structure in the context of your project. Do not be afraid to be verbose

2.3 Algorithms

2.3.1 AI Placing ships

When it is the AI's turn to place ships, the game class calls a getShipPlacement function within the AI class. In this function, the AI creates random x and y coordinates. The game class then attempts to place a ship there. If it can't then it calls the AI class again.

2.3.2 Validating ship locations

To check if a specific ship location and ship size is valid the board class runs the following algorithm:



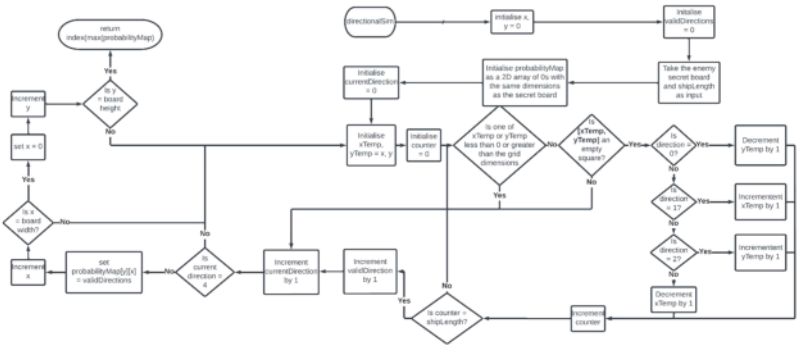
To check if there is a ship at [x,y] the board class checks if self._grid[y][x] = 1. If it is, then there is a ship at [x,y].

2.3.4 AI Attacking

There are two parts to attacking. Searching for ships and finishing off a ship. I have discovered two alternate algorithms for the searching part, and one for finishing ships.

Direction testing

The fastest way I could find for the AI to find a new ship to destroy.



This flow chart looks quite complicated, but the general idea is to iterate through every square in the grid and check how many directions you can place down a ship. For example, if you wanted to place a size two ship in the top left corner you could place it facing right or down, but not up or left, so the number of valid directions would be two. Then after the whole grid has been iterated through, the index of the square which allows for the most directions is returned. Then the square with that index would be attacked.

Brute force simulation

The second option for AI attacking would be to randomly simulate placing down ships on the enemy's secret board, then attack the square where the most ships have been placed.

```
def bruteForce(board, ships, width, height):
    countBoard = [[0] * width] * height
    for square in board:
        if square is not empty:
            add index(square) to taken
    simulationsCounter = 100000
    while simululationsCounter > 0:
        tempTaken = taken
        for shiplength in ships:
            while True:
                x = random.RANDOM_INT(0, width-1) # inclusive
                y = random.RANDOM_INT(0, height-1)
                dir = random.RANDOM_INT(0,3)
                if isValid(x,y,dir,shiplength):
                    for square covered by ship in countBoard:
                        square += 1
                        add index(square) to tempTaken
                    break from while loop

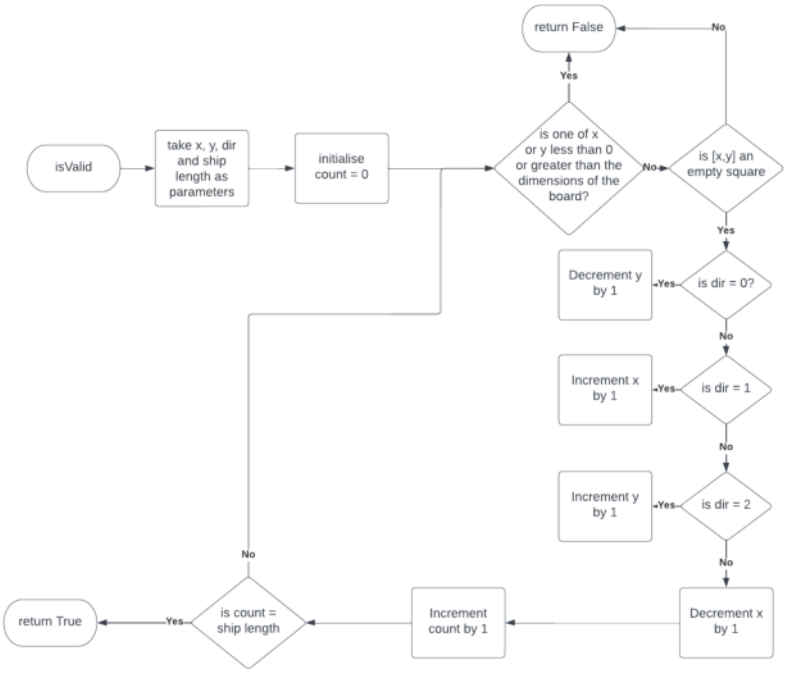
        simululationsCounter -= 1

    return index(max(countBoard))
```

The function takes the board, shipLengths array, width and height of the board as parameters. It then iterates through the board to check where the AI has already attacked. The squares that the AI has attacked are added to a taken list.
Then the simulations begin. Within one simulation, the algorithm creates random x and y coordinates, then a random direction. It then checks if it can place the ship at [x,y] facing in the direction corresponding to the dir variable. If it can, it adds 1 to all of the squares which the ship touches in the countBoard, and also adds those indexes to the tempTaken array. countBoard starts as a blank grid with the same dimensions as the secret board. By the end of the simulations shows exactly how many ships were in each square.

IsValid()

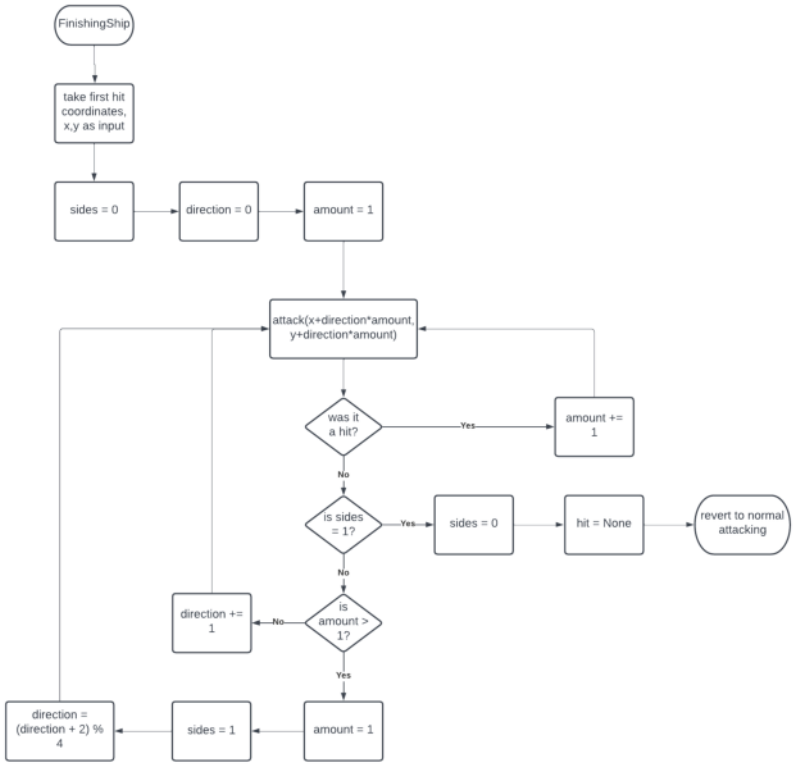
To check if the ship can be placed, the bruteForce function uses an isValid function. A flowchart for isValid is shown below:



Finishing a ship

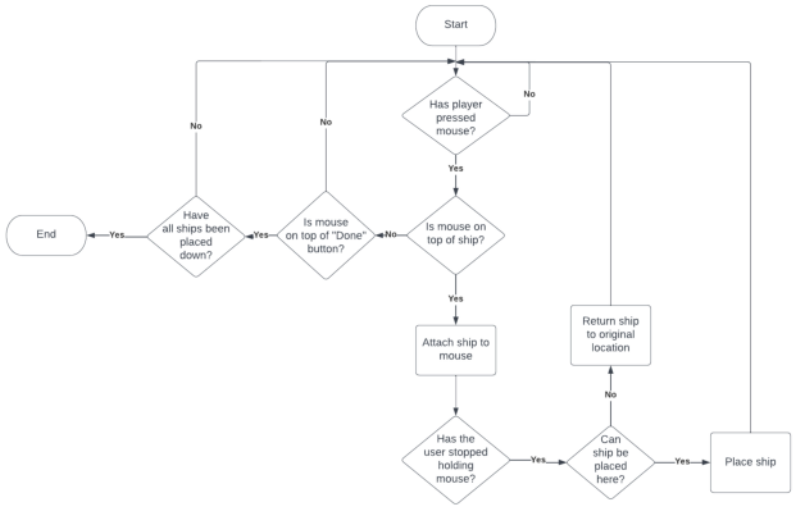
Upon hitting a ship, the AI should hit orthogonally adjacent squares until it gets another hit. Then it should continue hitting in that direction. If it misses, it should hit in the complete opposite direction from the square it first hit. When it has been told that the ship is destroyed, the AI should return to its normal attack algorithm.

Once the AI finds a ship using its regular method the following algorithm should be used:


FinishingShip
take first hit coordinates, x,y as input
sides = 0
direction = 0
amount = 1
attack(x+direction*amount, y+direction*amount)
was it a hit?
Yes
amount += 1
No
is sides = 1?
Yes
sides = 0
hit = None
revert to normal attacking
No
direction += 1
No
is amount > 1?
Yes
direction = (direction + 2) % 4
sides = 1
amount = 1

2.3.5 User placing ships in GUI

The player will be able to drag and drop ships onto the grid, so I will have to create an algorithm which can makes a ship follow the cursor around and then place it onto the square that is nearest to the pointer. Pressing R should also rotate this ship 90 degrees clockwise.



"Place ship" means that the ship image object will remain where the user placed it. However, the "addShip" function of the game class will only be called once the user has placed all their ships on the grid and clicked the "done" button.

2.3.6 User Attacking

For attacking in, a similar algorithm checking where the user's mouse is will be applied to find where the user attacked. Then the Board class will apply the following algorithm:

```
def attack(board, x, y):
    if board[y][x] has ship:
        mark board[y][x] as hit
        decrement totalHP counter
        if totalHP == 0:
            return won
        else:
            return hit
    else if board[y][x] has already been hit:
        return try again
    else:
         return miss
```

2.3.7 Hashing algorithm

This algorithm is used in the UI class. It receives a string and returns a unique "hash" of that string. It is used to store and transfer passwords securely. Whenever the UI has to send the users password to the

server, it instead sends a hashed version of the username and password.

```
def hash(toHash):
    if len(toHash) >= 100:
        return None
    else:
        while len(toHash)*2 <= 100:
            toHash = toHash + toHash
        count = 0
        while len(toHash) < 100:
            toHash = toHash + toHash[count]
            count += 1

    previous = ord(toHash[0]) XOR ord(toHash[-1])
    new = chr(previous)
    repeat 1000 times:
        for character in toHash:
            new_character = ((ord(character) XOR previous)%26)+65
            new = new + chr(((new_character XOR previous) % 26) + 65)
            previous = (new_character % 26) + 65
        toHash = new
        new = chr(previous)

    return toHash
```

While this code looks quite complicated, the general idea is to apply non-reversible mathematical operations onto a string so many times that the string looks like completely indistinguishable and seemingly random.

The first if statement checks if the inputted string is too long, if it is, nothing is returned and the UI interprets this as an invalid password and username combination. If it is not, then the string is doubled until doubling it would cause it to be too long. Then characters from its start are appended onto it until it has length 100. This guarantees that every hash in the database will have the same length.

The ord() function returns the Unicode integer of a character. The chr() function reverses this by returning the Unicode character which corresponds to a given integer. XOR performs a logical exclusive OR on two integers.
To hash a string, we have a pointer on a part of the string. We take the previous character of the string, XOR its ord() value with the ord() value of the current character, get the remainder of that when dividing by 26, then add 65*. Then update the current character to be the chr() of that value. Repeat this operation an arbitrary amount of times (in this case 1000) to get a seemingly random string, then return the remaining 100 length string.

* The remainder and addition are required because the ord() values for capital letters range from 65 to 90 inclusive, and we only want capital letters in our hash because I have found that other characters (e.g. "\") cause SQL errors in my database.

While I cannot guarantee that this is a perfect hashing algorithm, I believe that it is quite good, and certainly sufficient for the bespoke software my client requires. The requirements I had for my hashing algorithm during researching were:

- Output must be constant – Output is constant.
- Output must be unique – The best evidence I can provide for this is that the hash for a given input varies drastically, even for small changes. Because of this, and the fact that the size of the string always increases, I would say that there are very few inputs which provide the same output. I tested five different strings to show this:

```
a = hash("This is a test string of length 99 to make sure that the hash algorithm can deal with long strings.'
b = hash("This is a test string of length 99 to make sure that the hash algorithm can deal with long strings?'
c = hash("this is a test string of length 98 to make sure that the hash algorithm can deal with long strings");
d = hash("This is a test string of length 98 to make sure that the hash algorithm can deal with long strings");
e = hash("this is a test string of length 98 to make sure that the hash algorithm can deal with long strings!'
```

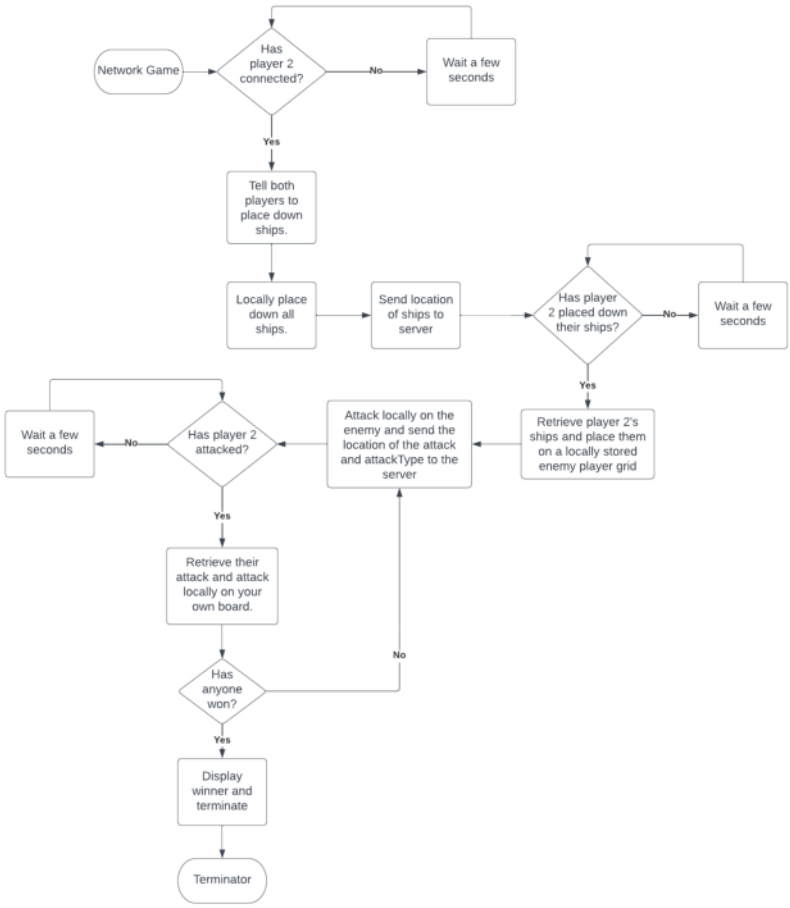
For each of these strings, I either changed to end punctuation or the capital 'T' at the start. This gave a result of:

```
a - NLGGLUGLYBPCUJFYNZAFSJCRFBALOFBNAKJLALIGCKFMIDABHJKDHDHGLNOAFFQFEQDONNSLAGHFDCCTJWKDNTXUFCZILBUTEMI
b - WVCCXUUFGDBEMRFQZNIDWJQBJFATAPZNWUHVCHCAKMXCMXCJXBCBLVLCTRIALLAJSCXCDFQNUEVNVEEJDQEPDVVURKVKNBCJQE(
c - EHEMDWCLOPNCURFETHCFEZADFDWHQFVVGTBFINYAEKTMQNENNPKTBLDCVTOMNJADAARYDRIJWCJBJACDPKGPNHJKHAHODJELOGI
d - OZKWFAADODPMYNJABVEXQTEDDJEBIXJDUBVPYJGAQCTOOBAZNRCDBBLSFVGOBJSFAUREDDAHEQNTXICVXACFFNJINEDSLXOZCUI
e - FTYCLACFWNNCEXNGHNAFWNGFFDAPWFZNAJBLMLSMWGBCCLQBRHSFBLTKBXGYLTGBAEFCDZMFAIVJBUIVFGEXVTFUBAFOLDMHMEI
```

- Impossible to reverse – This is also difficult to prove, however due to the use of XORs, we are losing information as we turn two operands into one result. This means that the result could have come from many different operands, therefore reverse engineering to get the correct operands should be very difficult, especially given the amount of XORs (99*1,000 = 99,000).

2.3.8 Network game

An online game will be very similar to a regular game, except both players will have to update and query the database stored on the server.



Pseudocode or similar for key algorithms essential to the success of the project. These MUST be linked into the how they fit into the project as a whole. Show how the algorithm would **affect** the **data structures** used in the context of your project. Give a **trace table/dry run** of sections of your key algorithms to **show** how they work.

2.4 File structure and organisation



The images will be stored in different folders depending on their scale. I will first make all the image components in the smallest scale I can as pixel art. This is because small art is the easiest and fastest to make, and provides no disadvantage in changing scale. Using a python script that I made with the PIL module, the program will scale these images up to the scale the user wants. The final game file will have two pre-installed scale folders, 1920*1080 and the original small-scale images in 96*54. During runtime the user will be able to create additional resolutions for the game by going into the options menu.

The scale folders and all other files/folders will be stored in a primary folder called Battleships.
Python files will be stored in a folder called “scripts”.
These python files include:

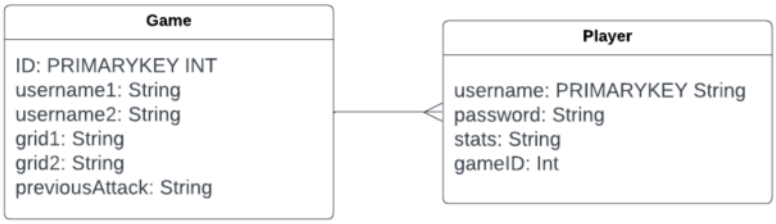
- Battleships.py which is used to run the main program and allows the user to choose between GUI and Terminal modes.
- Board.py, Game.py, Player.py, UI.py and AI.py contains definitions for their respective classes. The UI.py file also contains definitions for the Terminal and GUI classes.

There will also be a lone options.txt file stored in the NEA folder. This file will store information about all of the options which the user has changed (for example, grid dimensions, number of ships, special attacks etc).

Separate to this NEA folder, there is a server folder stored on my laptop. This folder has a server.py file and a battleships.db file which stores the database.

Show what files are going to be created and which software objects will be implemented in each file. Include folder structure for images, database file if used.

2.5 Database design



2.5.1 Game Table

The Game table is used for both saving games and network games. When a network game is created or a game is saved, a row is added to the Game table. That row has a unique ID, two player usernames, two strings containing ship locations and a previousAttack string. The ship locations are stored as long strings of "|" separated coordinates separated by commas. For example, if there is a ship at [2,2] and [5,5], the ship string would be "2,2|5,5".
The previousAttack string is only needed for network games. It stores the most recent attack to allow the other client to know where they have been attacked. This is stored as x y attackType playerNumber (separated by spaces).

2.5.2 Player Table

The player table is used to store statistics which my user wanted and saved Game IDs. These are comma separated strings. The player table also stores the players username and a hash of the password.

Provide schema for database. Give example data that would be stored in the database for your project.

2.6 SQL queries

In order to allow my clients to use databases, I will have a server script running on my laptop. I will also store an SQL database on that laptop.

Game saves/loads

When loading games, the user will input a GameID, and then the UI class will send a request through the network class to the server. The server will then query the database:
Select shipOne, shipTwo FROM Game WHERE GameID = userinput

When saving games an insert command is required.
INSERT INTO Game
VALUES (id, username, grid1, grid2)

User Accounts

The user will input a username and password. The UI will hash a combination of the username and password, then send a request containing the username and hashed combination through the network class to the server. The hashed combination will be stored as the password in the database. The server will query the database:
SELECT Password FROM Player WHERE Username = username
Only hashed versions of the password will be stored in the database. After retrieving the password, the server will check that the hashed passwords match, and if they do, the server will send a success message, and the player will be able to login.

When the user is signing up, a similar process happens. However, this time the server will have to check that the username is unique, and then it can add it to the database.

INSERT INTO Player
VALUES (username, password)

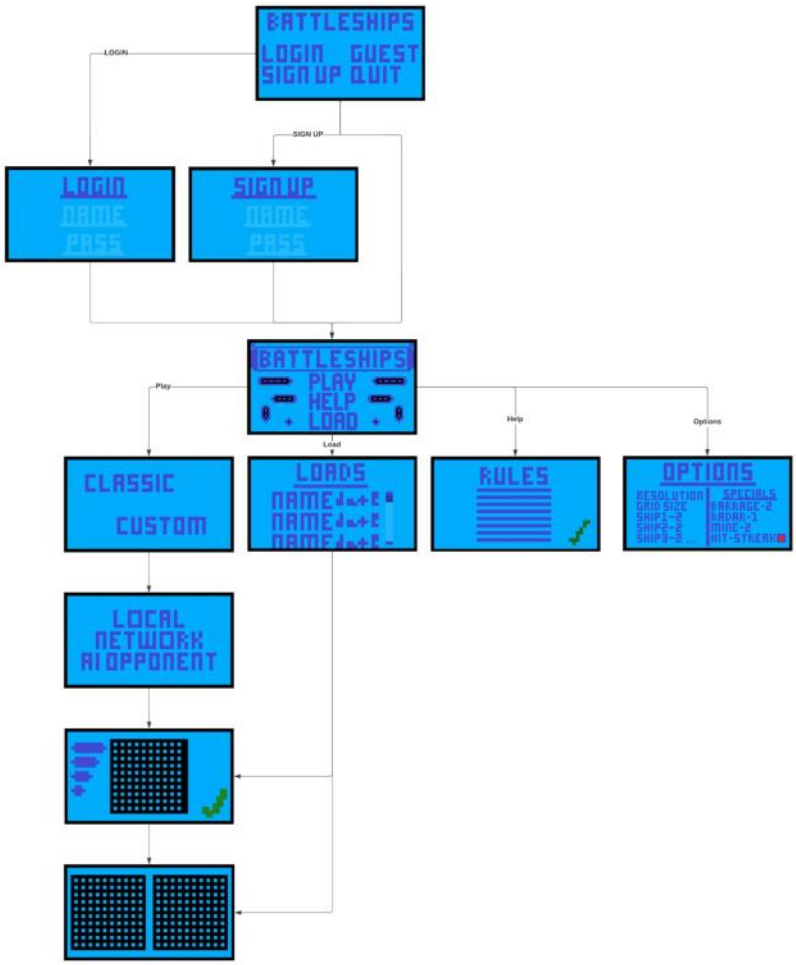
Network Games

When saving a network game or a previously saved game (a game that already has a row in the Game table) an update command will be required:
UPDATE Game SET shipOne = new_ships WHERE ID = gameID
or
UPDATE Game SET shipTwo = new_ships WHERE ID = gameID

Each time a player attacks, the server will have to update the recentAttack column of the game.
UPDATE Game SET recentAttack = newAttack WHERE ID = gameID
The server will also have to query the database to get the recent attack each time to get the recentAttack back for the player being attacked:
SELECT recentAttack FROM Game WHERE gameID = gameID

Give **all** SQL queries that would be needed to implement specific project objectives. (can be written after solution completed)

2.7 User interface design



Provide wire-frame diagrams of the graphical user interface, positioning of buttons, different screens. Give any menu commands that would be available in a terminal interface

2.8 User guide

To run this program, the user will need to have a python IDE. They will also need to install the pprint, os, pygame and art modules for python.

Once all of this is ready, the user should navigate to the file path where Battleships.py is stored through a terminal using the “cd” command.

Then the user can write: “python Battleships.py g” or “python Battleships.py t” depending on whether they wish to play the terminal (t) version or the GUI (g) version.

If the user requires help on how to play the game of Battleships, they should choose the “help” option in the main menu.

Write a user guide to say what software or modules the user will need to install or have available on their system, and how they should run your program from the command line

Examiners report – an important read!
It was pleasing to see good students carefully structure out the design of their technical solution. Effective use of diagrams to provide an overview of the whole system, key data requirements being identified and explained along with a breakdown of the complex parts leading to pseudo-code and/or code snippets would lead to a high mark. It was also common, however, to see a more random attempt at the design documentation including just pasting code across with no detail as to the design process or how it would link into the main system. So, for example, just providing stock algorithms for merge sort and binary search does not help the reader understand the design of the system.
Having a section titled ‘sample of SQL queries’ is not very beneficial in providing a reader an understanding as to how the system will work. Students would do better to design out a particular form/page and then discuss the algorithms required for that part of the system including the SQL queries to be used for that part. Students should be encouraged to think about the data to be used by the system. In a quiz system, for example, it would be beneficial to provide examples of the kind of question(s) to be asked. For a simulation it would be good to see how the formulas are to be used alongside, for example, a sketch of the trajectory of the projectile being modelled.
For a game a student could sketch out the grid or level and talk through, for example, the movement of any enemies.

It was common to see algorithms appear without the reader having any real understanding as to how these fitted into the system and a few sketches or examples of the data to be used would help. So, for example, one student produced an excellent Sudoku solver which had some complex pseudo-code in the design section. This code was hard to understand but a few sketches of particular board layouts showing how the individual functions would perform would have really helped. It was common to see many students make use of well-known algorithms such as the merge sort. Just providing the pseudo code for this algorithm is not going to help their documented design mark. If the student talks about how this algorithm is going to be used by the system and integrated then this is beginning to pick up some credit. If the only pseudo-code or algorithm design a student attempts is based around merge sort, quick sort, binary search or other well-known algorithms without any attempt at looking at other parts of the system then the student should not be scoring highly in the documented design section.

3 Technical Solution *(42 marks)*

3.1 Completeness Section

Below is a table describing how well I completed each of the objectives from section **1.5 Numbered measurable, appropriate specific objectives of the project.**

Number	Objective	Not, partially, or entirely completed.	Timestamp in video.
Stage 0: Basic Terminal			
1	The program should be runnable with a 't' option to run in the terminal.	Entirely. By writing "python scripts\Battleships.py t" the terminal game starts.	
2	At the start of the game, a two-dimensional array should be displayed and four grid references to place down a ship in the format 'x,y,v'/'x,y,h' should be accepted. The x gives the left most square of a ship, and the y gives the topmost square of a ship, v/h are for vertical/horizontal. This needs to be repeated for both players.	Entirely. local(), networkGame() and getShipInput() in the Terminal class.	
3	The locations of ships should be stored in a two-dimensional array.	Entirely. grid and sGrid are two-dimensional arrays in the Board class.	
4	At the start of a turn, the players previous attacks and current ships will be printed in the terminal.	Entirely. local() and networkGame() functions in the Terminal class.	
5	One grid reference indicating where to attack in the format 'x,y' will be accepted from the user.	Entirely. getAttackInput in the Terminal class.	
6	After each turn input will pass to the other player and information about their current ships and previous attacks should be presented.	Entirely. local() and networkGame in the Terminal class.	
7	When one side has no squares with an intact ship, the game should end and declare the winner.	Entirely. attack() in the Board class.	
Stage 1: Full Terminal			

8	Running the program without a 't' option should display an error.	Entirely. The user can now also use a 'g' option, but otherwise an error message is shown. See Battleships.py file.	
9	When placing ships, the user should not be allowed to place them so that ships overlap or do not fit in the grid.	Entirely. addShip in the Board class and local() and networkGame() in the Terminal class	
10	When attacking, the user should not be allowed to attack a square they have already attacked.	Entirely. attack() in Board class and local() and networkGame() in the Terminal class	
11	When attacking, the user should not be allowed to attack a square outside of the grid.	Entirely. attack() in Board class and local() and networkGame() in the Terminal class	
12	After a turn is over, the terminal should be cleared, preventing the second user from seeing information about the first users ships and attacks.	Entirely. local() and networkGame() in the Terminal class	
13	There should be more than one game mode.	Entirely. Instead of multiple game modes, the user is allowed to modify all options to make their own game mode. See options(), local() and networkGame()	
14	Before the game starts, the user should be able to decide the grid size, number of ships, ship types, special attacks and hints.	Entirely. When playing online on different devices, they can individually change their options. Playing locally, both users have the same options.	
15	A game mode which allows more than two players to play against each other.	Entirely. local() in Terminal class	
Stage 2: Basic GUI			
16	Running the program with the 'g' option should run the GUI version of the game, displaying "Help", "Play", and "Quit" options.	Entirely. Battleships.py file and menu() in GUI class.	
17	These options should be selectable by moving and clicking with a mouse.	Entirely. menu() in the GUI class	
18	Upon selecting the "Quit" option, the GUI should close, and the program should halt.	Entirely. menu() in the GUI class	

19	Selecting the “Play” button should display a new GUI showing different game mode options.	Entirely. menu() calls playMenu() in the GUI class	
20	Pressing the escape key or exit option should close this new GUI, returning to the main menu.	Entirely. playMenu() in the GUI class	
21	In the Play window, choosing the classic mode option should start a new game, displaying a GUI for placing down ships.	Entirely. playMenu() calls a local game.	
22	The user should be able to drag and drop ships from a list onto a 10x10 grid.	Entirely. local() and networkGame() in the GUI class.	
23	Upon pressing the ‘R’ key, the most recently selected ship should rotate 90 degrees.	Entirely. local() and networkGame() in the GUI class.	
24	The user should be able to press a “Ready” button, allowing the second user to place down their ships.	Entirely. local() and networkGame() in the GUI class	
25	The second user should be given the same blank grid and ships the first user was given.	Entirely. local() and networkGame() in the GUI class	
26	The locations of both users’ ships should be stored in two different two-dimensional arrays.	Entirely. See init() and addShip() in the Board class.	
27	Once the second user has pressed the “Ready” button, a new GUI prompting the first user to attack should be shown.	Entirely. local() and networkGame() in the GUI class	
28	The first user should be able to select a square on the grid to attack.	Entirely. local() and networkGame() in the GUI class and attack() in the Game, Player and Board classes.	
29	While thinking of where to attack, the user should be shown two grids. One with their previous attacks, and another with all of their current ships and their state.	Entirely. local() and networkGame() in the GUI class	
30	Upon attacking, the other user should be prompted to attack.	Entirely. local() and networkGame() in the GUI class	

31	After each attack, the program should check if the attack hit, and updating the two-dimensional array containing the other user's ship's if it hit.	Entirely. attack() in the Board class.	
32	This cycle should continue until all ships in one two-dimensional grid have been fully destroyed.	Entirely. attack() in the Board class checks whether all ships have been destroyed, local() or networkGame() keep the loop going and declare the winner.	
33	The loser is the one who's two-dimensional grid containing ships is first destroyed.	Entirely. local() or networkGame() in the GUI class declare the winner.	
34	The winner should be displayed, and the GUI should return to the main menu once someone has won.	Entirely. local() and networkGame() in the GUI class.	
35	The user should be able to change the resolution of the window.	Entirely. addNewResolution() and options() in the GUI class.	
Stage 3: Full GUI			
36	In menus, things should be selectable using either the mouse or keyboard presses.	Entirely. The user is able to fully navigate around the game using mouse/key presses. (i.e. no terminal input required)	
37	When turns change, the program should change to a blank screen, then after any input from the user change to the other users board.	Entirely. Using a "posthit" Boolean in the networkGame() and local() functions the game knows whether or not to display the boards.	
38	Pressing the "Help" button should open a menu displaying information about battleships and how this version of battleships works.	Entirely. Pressing a magnifying glass in the play menu leads to the help screen. See playMenu() in the GUI class	
Stage 4: Full Game			
39	An additional "Options" button should be added to the main menu.	Entirely. The user can click a cog in the play menu to go to options. See playMenu() in the GUI class	
40	Upon pressing the options button, an options menu should appear, showing	Entirely. options() in the GUI class.	

	the different things the user can change.		
41	The user should be able to customise the resolution.	Entirely. options() can call the addNewResolution() function, both in the GUI class.	
42	The user should be able to customise what the GUI looks like. Mainly the colour scheme.	Entirely. The user can change the colour scheme in options()	
43	An additional game with different attacks should be included.	Entirely. Not implemented as a pre-defined game mode, but the user is allowed to change all of the important game options to make their own game modes, including different attacks. See options() in the GUI and Terminal classes.	
44	The user should be able to select this mode after pressing the “Play” button in the main menu.	Entirely. The user can first change their options, then press Play to play a game with those settings.	
45	In this game mode, before it starts, the user should be able to choose which attacks should be included in the game.	Entirely. options() in the GUI and Terminal classes.	
46	These attacks include barrage, mine, and radar.	Partially. Both radar and barrage are include. See options().	
47	The user should also be able to toggle a “Hit Streak” button which allows the player to attack again if they hit a ship in the special mode.	Not done.	
48	Barrage attacks multiple squares.	Entirely. attack() in the game class attacks 9 different squares if it is called with a barrage parameter.	
49	Mines should be placed down by the user while they place their own ships, and if an enemy hits a mine during gameplay, a ship on the enemy’s square takes one damage.	Not Done	
50	Radar finds the location of an enemy ship.	Entirely.	

		radar() in the Game class loops through the enemies board and returns the first ship location it finds.	
Stage 5: AI Opponent			
51	Upon selecting a game mode, the user should be able to select to either play against AI or another player locally.	Entirely. The playMenu() function in the GUI and Terminal classes asks whether the user wants to play against AI. AI.py contains the AI class which acts as a player in a game.	
52	The user should be able to choose between three different difficulty levels of AI.	Entirely. playMenu() in the GUI and terminal classes.	
53	The harder AI should always attack the most likely square where an enemy ship may be.	Entirely. getAttack() and chooseAttackSquare() in the ai class behave differently depending on the AI's difficulty.	
54	The easier AI should attack more randomly, with a weighted probability of attacking better squares.	Entirely. getAttack() and chooseAttackSquare() in the ai class behave differently depending on the AI's difficulty.	
55	Upon hitting a ship, all AI should hit clockwise around the hit ship to find where it continues, and then shoot in that direction until the ship ends.	Entirely. Using the getHitFeedback the AI knows where to attack if it has hit a ship on the previous turn(s).	
56	Once the AI misses in that direction and the ship is not yet destroyed, the AI should attack the opposite side of the original hit.	Entirely. See getAttack() and hitFeedback() in the AI class.	
Stage 6: Multiple Players			
57	An additional option allowing for upto 4 players to play against each other.	Entirely. In the options menu the user can customise the number of players. Any number of players greater than 1 is allowed.	
58	During each turn, a player attacks all other players.	Entirely. local() in the GUI and Terminal classes loop "numPlayers" when a player is attacking.	
59	A player is knocked out when their ships have been destroyed.	Entirely.	

		local() in the GUI and Terminal classes use a list and numPlayers variable to keep track of which players remain.	
60	They should no longer have a turn, and no one should be able to attack them anymore.	Entirely. The eliminated player is removed from the list. See local() in the GUI and Terminal classes.	
61	The winner is the last remaining player.	Entirely. local() in GUI and Terminal classes.	
		Stage 7: Save Games	
62	In the middle of a game, the user should be able to press a button to save and quit the game.	Entirely. Inputting “s” or pressing the “s” key in the terminal or GUI version respectively saves and quits the game. See save() and local() in the GUI and Terminal classes	
63	Upon saving a game, the state of the game should be uploaded to a central database stored on a server.	Entirely. Using the network, the save() function in the GUI or Terminal class calls saveGame() in the server class.	
64	In the main menu, an additional “Load Game” button should be added.	Entirely. playMenu() calls load() in the GUI and Terminal classes.	
65	This menu should display information about save games and/or the ID which allows the user to load them.	Partially. The game ID’s are displayed but information about the game isn’t. See load() in the GUI and Terminal classes.	I will fix this before the final deadline.
66	If the user selects a save game, the user should be sent to that game to play against the correct opponent.	Partially. Loading a game continues that game as a local game, however not necessarily against the correct human opponent. However, it will remember the correct AI. See load() in the GUI and Terminal classes.	
67	The save game must contain the locations of both sets of ships, the type of opponent, and which player owns the respective states.	Entirely. A game stores the gameID, grid1, grid2, username1, username2 and previousAttack. This information is parsed in the load() function of the GUI and Terminal classes.	
68	The user should be able to leave the Load Game	Entirely. Pressing Escape in the GUI or writing “E” in the terminal returns to the playmenu.	

	menu by pressing the Escape key.	See load() in the GUI and Terminal classes.	
Stage 8: User Accounts			
69	When first opening the game, the user should be given the option to play as guest, log in or register.	Entirely. The menu() function in the GUI and Terminal classes allows the user to go to a login or signup screen or immediately to the playMenu (as guest).	
70	The user should be able to play the game as a guest or with an account.	Entirely.	
71	Logging in should allow the user to input a username and password.	Entirely. See authenticate() in the GUI class or logIn() and signUp() in the Terminal class.	
72	The stored password should be encrypted.	Entirely. A password and username combination is run through a hashing algorithm before the details are sent to be stored at the server.	
73	If the username and password match, the user should be let into their account.	Entirely. If the server returns a “valid” response, the user is let into their account.	
74	Look-up time for passwords should be quick.	Entirely. According to the time.perf_counter() function from the time module, average lookup is roughly 0.0188 seconds.	
75	Pressing the sign-up option should allow the user to create a new account with a new username and password.	Entirely. See signUp() or authenticate() in the GUI and Terminal classes. The playMenu() function calls these functions.	
76	The user should only be allowed to make a new account if the username is unique.	Entirely. The server checks if the username is unique, if it isn’t an “invalid” response is returned, and the signUp() or authenticate() function queries the user again.	
77	After registering the user should immediately be logged into that account.	Entirely. After signing up the user goes to the play menu.	
78	Account information should be stored in a database on a server.	Entirely.	
79	An account should keep track of various statistics	Entirely.	

	of the player and the matches they have played.	See stats() in the GUI and Terminal classes. Also getStats() and updateStats() in the Server class.	
80	Account information should include total win rate, least attacks against AI and best number of ships left against AI and overall hit accuracy.	Entirely.	
81	When logged in, a user should be able to store saves in a database on a server.	Entirely. In the middle of the game, the user can call the save() function which calls the server and saves the required information onto the database.	
82	When logging in from different devices, the user should be able to see all of the stored statistics and save states.	Entirely. See save() and stats() in the GUI and Terminal classes.	
83	In the main menu, a new option called “Statistics” should exist, showing all of the information about this account.	Entirely. See stats() and mainMenu() in the GUI and Terminal classes.	
84	The user should be able to return back to the main menu by pressing the escape key or quit button while in the settings menu.	Entirely. See stats() and mainMenu() in the GUI and Terminal classes.	
85	When playing an online game, if both players are logged in, the game should check if one player seems far better than the other	Entirely. networkGame() in the GUI and Terminal classes and compareStats() in the UI class.	
86	This can be done by checking the amount of wins against each other, and the total amount of games played, hit accuracy wins against AI etc	Partially. compareStats() in the UI class compares their total win rates and hit rates, but not against each other.	
87	If this is the case, the game should suggest that the worse player is given an advantage	Entirely The players are informed of this, and suggested to go to the options menu and make it easier for that player.	
Stage 9: Network Games			
88	Two players should be able to play against each	Entirely.	

	other on different devices by connecting to a server	Choosing the online option calls a networkGame() function which connects both players to a server.	
89	To start a network game, both players must be logged in and input the username of their opponent	Entirely. See networkGame() function in the GUI and Terminal classes	
90	Once a game has started, players should be able to place ships simultaneously.	Entirely. See networkGame() function in the GUI and Terminal classes.	
91	During attacking, players should be able to attack simultaneously.	Entirely. See updatePreviousAttack() in the Server class and the networkGame() function.	
92	The difference between the number of attacks performed by players should be at maximum one.	Entirely. The server waits for the other player to attack before consider the first players attack if the first player is ahead. See networkGame() in the GUI and Terminal classes and updatePreviousAttack() in the Server class.	
93	Everything which can be done in a local game, should be possible in an online game.	Partially. Everything works fully except for saving games and more than two player games	

In the completeness section we are looking for a consideration as to:

- Has the project met the objectives set by the student in the analysis stage?
- How well have these objectives been met (consideration to HCI / features)?
- Does the technical solution match the original project background description?

Technical solution – completeness		
Level	**Criteria**	**Mark**
3	A system that meets almost all of the requirements of a solution/an investigation (ignoring any requirements that go beyond the demands of A-level).	11-15

3.2 Technical Skills Section

3.2.1 Overview Guide

Level	Name	Description	Where implemented
A	Hashing algorithm	Returns a unique, unreadable (for humans) string of length 100 from a given password.	hash() in the UI class
A	Inheritance	The AI class inherits from the player class. The GUI and Terminal inherit from the UI class	AI, GUI and Terminal classes.
A	Composition	The UI creates a game which has two players. Both players have a board.	Used in the UI, Game, Player and Board classes to pass information down from the user.
A	Lists	Lists are used to keep track of ships and their locations and size.	Used in the UI, Game, Player, and Board classes to keep track of ships and the game rules.
A	Client server model and server-side scripting	User input is used to connect to a server through a network. This server runs functions which query an SQL database stored on the server.	UI class uses the network in networkGame(), signUp(), logIn(). The network starts a thread() in the server.py file.
A	Complex user-defined algorithm	An AI which can determine the best location to place its ships, and where to attack the enemy	AI class in ai.py. getAttack(), getShipPlacement() and chooseAttackSquare()
B	Single table parameterised SQL	An SQL database contains a Game and Player table. These are used to allow the user to save and load games using their account.	Every function except threading() and startGame() in the server.py file uses SQL.
B	Multi-dimensional array	Game boards are stored as two-dimensional arrays.	grid and sGrid are two-dimensional arrays in the Board class in the Board.py file.

B	Text files and reading/writing from text files.	User options are stored in a text file. When the program starts, this file is read and when options are changed, the file is written onto.	UI class in UI.py file. In the options() function.
B	Simple data model in database.	A games table and player table.	Used in the SQL database stored on the server.
B	Dictionaries	Used to store the option that should be changed and what it should be changed.	options() in the GUI and Terminal class

Technical solution – techniques used		
Level	**Criteria**	**Mark**
3	The techniques used are appropriate and demonstrate a level of technical skill equivalent to those listed in Group A in **Table 1**. Program(s) demonstrate(s) that the skill required for this level has been applied sufficiently to demonstrate proficiency.	19-27

In the technical skills section, we are looking for a marker to:

- Identify parts of the code where complexity is clearly evident and map to Group A/B/C **[This can be helped by a student producing an overview guide]**
- Place consideration into the coding style and comment upon this
- Consider the overall effectiveness of the final solution (does it work how it should?)

You should produce a guide here that highlights techniques/ skills in sections A B and C with commentary. Your code should be commented to show where specific technical skills are being used e.g. # SKILL GROUP A – Graph Traversal

3.2.2 Code listing

```
from abc import abstractclassmethod
from Game import Game
from pprint import pprint as pp
import os
import random
import pygame as pg
from art import *
from network import Network
from PIL import Image

class UI():
    def __init__(self):
        self._username = ""
        with open("options.txt", "r") as f:
            data = f.readline()
            data = data.split(",")
        data = list(map(int, data))
        Game.changeBoardDimensions(data[0], data[1])
        for index, number in enumerate(data[2:8]):
            Game.changeShips(index, number)
        for index, number in enumerate(data[8:13]):
            Game.changeSpecials(index, number)
        Game.changeNumPlayers(data[13])
        Game.updatePlayers()

        ################################################################################
        #
        # CATEGORY A SKILL: Composition
        # Defining a self._network variable which is a Network object. UI has a Network.
        #
        ################################################################################
```

```
        try:
            self._network = Network()
            pass
        except:
            print("Could not connect to server")

    @abstractclassmethod
    def __run():
        raise NotImplementedError

    @abstractclassmethod
    def __local():
        raise NotImplementedError

    @abstractclassmethod
    def __networkGame():
        raise NotImplementedError

    @abstractclassmethod
    def __help():
        raise NotImplementedError

    @abstractclassmethod
    def __load():
        raise NotImplementedError

    @abstractclassmethod
    def __save():
        raise NotImplementedError

    ################################################################################
    #
    # CATEGORY A SKILL: HASHING
    # Using a hashing algorithm to securely transfer user passwords.
    #
```

```
    ###########################################################################
    def hash(self, toHash):
        length = 100
        if len(toHash) >= length:
            return None
        else:
            while len(toHash)*2 <= 100:
                toHash += toHash
            toHash += toHash[:100-len(toHash)]
            print(len(toHash))

        prev = ord(toHash[0]) ^ ord(toHash[-1])
        new = chr(prev)
        for _ in range(1000):
            for ch in toHash[1:]:
                ch = ((ord(ch)^prev)%26)+65
                new += chr(((ch^prev) % 26) + 65)
                prev = (ch % 26) + 65
            toHash = new
            new = chr(prev)
        print("DEBUG -- hash complete")
        return toHash

    def compareStats(self, s1, s2): # wins,losses,hits,misses,highestHP,leastAttack INTEGER LISTS
        stats = [s1,s2]
        winrates = []
        hitrates = []
        for statList in stats:
            if statList[0] == statList[1] == 0:
                winrates.append(0)
            else:
                winrates.append(statList[0]/(statList[0]+statList[1]))

            if statList[2] == statList[3] == 0:
                hitrates.append(0)
```

```
            else:
                hitrates.append(statList[2]/(statList[2]+statList[3]))

        if abs(winrates[0]-winrates[1]) > 0.1:
            if winrates[0] < winrates[1]:
                return 0
            else:
                return 1
        if abs(hitrates[0]-hitrates[1]) > 0.1:
            if hitrates[0] < hitrates[1]:
                return 0
            else:
                return 1
        return -1


    ##############################################################################
    #
    # CATEGORY B SKILL: Text files and reading/writing from text files
    # Reading a text file and then updating its contents based on a dictionary.
    #
    ##############################################################################
    ##############################################################################
    #
    # CATEGORY B SKILL: Dictionaries
    # Using a dictionary to know which options to change, and what they should
    # be changed to.
    #
    ##############################################################################
    def updateOptions(self, changed):
        Game.updatePlayers()
        with open("options.txt", "r") as f:
            data = f.readline().split(",")
        for index, newValue in changed.items():
            data[index] = str(newValue)
```

```
        with open("options.txt", "w") as f:
            f.writelines(",".join(data))

################################################################################
#
# CATEGORY A SKILL: Inheritance
# The GUI inherits from the base UI class.
#
################################################################################
class GUI(UI):
    def __init__(self):
        super().__init__()
        self._scale = 20
        self._width = 48*self._scale
        self._height = 27*self._scale
        self._finalImagePath = f"FinalImages\\Scale{self._scale}\\"
        self._fillColour = (0,168,243)

    def run(self):
        self.__mainMenu()

    def __stats(self):
        if self._username != "":
            stats = self._network.send(f"getStats {self._username}") # Returns comma seperated string
        else:
            return
        letters = []
        for character in stats:
            letters.append(pg.image.load(f"{self._finalImagePath}{character}.png"))

        alphabet = []
        alphabet = [pg.image.load(f"{self._finalImagePath}{ch}.png") for ch in "abcdefghijklmnopqrstuvwxyz"]
        titles = ["wins", "losses", "hits", "misses", "maxhp", "attacks"]
```

```
pg.init()
screen = pg.display.set_mode((self._width, self._height))
runningStats = True
camera = 0
clock = pg.time.Clock()

while runningStats:
    for event in pg.event.get():
        if event.type == pg.QUIT:
            pg.quit()
        elif event.type == pg.KEYDOWN:
            if event.key == pg.K_ESCAPE:
                runningStats = False
        elif event.type == pg.MOUSEWHEEL:
            camera += event.y*30
    clock.tick(60)


    screen.fill(self._fillColour)
    xposition = 0
    letterSize = 3
    yposition = self._scale
    for ch in titles[0]:
        screen.blit(alphabet[ord(ch)-97], (xposition, yposition + camera))
        if ch == "m" or ch == "w":
            letterSize = 6
        elif ch == "q":
            letterSize = 5
        elif ch == "i":
            letterSize = 2
        else:
            letterSize = 4
        xposition += self._scale * letterSize
    xposition += self._scale * letterSize
    i = 1
```

```
            for index, letter in enumerate(letters):
                if stats[index] == ",":
                    yposition += 6*self._scale
                    xposition = 0
                    for ch in titles[i]:
                        screen.blit(alphabet[ord(ch)-97], (xposition, yposition + camera))
                        if ch == "m" or ch == "w":
                            letterSize = 6
                        elif ch == "q":
                            letterSize = 5
                        elif ch == "i":
                            letterSize = 2
                        else:
                            letterSize = 4
                        xposition += self._scale * letterSize
                    xposition += self._scale * letterSize
                    i += 1
                else:
                    screen.blit(letter, (xposition, yposition+camera))
                    xposition += self._scale*3
            pg.display.update()

    def __mainMenu(self):
        # Initialise
        pg.init()
        screen = pg.display.set_mode((self._width, self._height))
        runningMenu = True

        # Buttons
        rects = []
        buttonImages = [pg.image.load(f"{self._finalImagePath}LoginButton.png"),
pg.image.load(f"{self._finalImagePath}SignUpButton.png"), pg.image.load(f"{self._finalImagePath}GuestButton.png"),
pg.image.load(f"{self._finalImagePath}QuitButton.png")]
        backgroundImage = pg.image.load(f"{self._finalImagePath}MainMenu.png")
        for image in buttonImages:
```

```
            rects.append(image.get_rect())
        # Position buttons correctly.
        rects[0].update(int(round((5/96)*self._width)), int(round((19/54)*self._height)), rects[0].width, rects[0].height)
        rects[1].update(int(round((5/96)*self._width)), int(round((38/54)*self._height)), rects[1].width, rects[1].height)
        rects[2].update(int(round((53/96)*self._width)), int(round((19/54)*self._height)), rects[2].width, rects[2].height)
        rects[3].update(int(round((53/96)*self._width)), int(round((38/54)*self._height)), rects[3].width, rects[3].height)
        rects.append(pg.Rect(41*self._scale, 22*self._scale, 6*self._scale, 4*self._scale))


        while runningMenu:
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    runningMenu = False
                elif event.type == pg.KEYDOWN:
                    if event.key == pg.K_ESCAPE:
                        runningMenu = False
                elif event.type == pg.MOUSEBUTTONDOWN: # Check if player pressed mouse
                    if event.button == 1: # Check if player pressed left mouse button
                        point = pg.mouse.get_pos()
                        for i, rect in enumerate(rects):
                            if rect.collidepoint(point):
                                if i == 0:
                                    self.__authenticate("login")
                                elif i == 1:
                                    self.__authenticate("signup")
                                elif i == 2:
                                    self.__playMenu()
                                elif i == 3:
                                    runningMenu = False
                                elif i == 4:
                                    self.__stats()

            screen.blit(backgroundImage, (0,0))
            for image, rect in zip(buttonImages, rects):
                screen.blit(image, rect)
```

```
            pg.display.update()

    def __playMenu(self):
        # Initialise
        pg.init()
        screen = pg.display.set_mode((self._width, self._height))
        runningPlayMenu = True

        # Buttons
        rects = [None for _ in range(7)]
        backgroundImagePlayMenu = pg.image.load(f"{self._finalImagePath}playMenu.png")
        rects[0] = pg.Rect(3*self._scale, 11*self._scale, 19*self._scale, 5*self._scale)
        rects[1] = pg.Rect(24*self._scale,11*self._scale,21*self._scale,5*self._scale)
        rects[2] = pg.Rect(3*self._scale, 18*self._scale, 5*self._scale, 5*self._scale)
        rects[3] = pg.Rect(37*self._scale,20*self._scale,3*self._scale,5*self._scale)
        rects[4] = pg.Rect(31*self._scale,20*self._scale,5*self._scale,5*self._scale)
        rects[5] = pg.Rect(12*self._scale,18*self._scale,15*self._scale,5*self._scale)
        rects[6] = pg.Rect(41*self._scale,20*self._scale,5*self._scale,5*self._scale)

        while runningPlayMenu:
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    runningPlayMenu = False
                elif event.type == pg.KEYDOWN:
                    if event.key == pg.K_ESCAPE:
                        runningPlayMenu = False
                elif event.type == pg.MOUSEBUTTONDOWN:
                    if event.button == 1:
                        # Check which menu button it collided with and call that function.
                        point = pg.mouse.get_pos()
                        for i, rect in enumerate(rects):
                            if rect.collidepoint(point):
                                if i == 0:
                                    print("Starting regular game")
```

```
                                self.__local(False, 0)
                            elif i == 1:
                                print("Starting online game")
                                self.__local(True, 0)
                            elif i == 2:
                                aiDifficulty = self.__getUserInput("aidifficulty")
                                try:
                                    aiDifficulty = int(aiDifficulty)
                                except ValueError:
                                    break
                                self.__local(False, aiDifficulty)
                            elif i == 3:
                                self.__help()
                            elif i == 4:
                                self.__options()
                            elif i == 5:
                                self.__load()
                            else:
                                runningPlayMenu = False
                                pg.quit()
                    screen.blit(backgroundImagePlayMenu, (0,0))
                    pg.display.update()

    def __local(self, network, ai, game=None): # Network is a boolean
        # Initialise
        pg.init()
        Game.updatePlayers()
        Game.changeAI(ai)
        screen = pg.display.set_mode((self._width, self._height))
        runningLocal = True
        currentPlayer = 0
        shipHP = Game.getShips()
        numships = Game.getNumShips()
        ##############################################################################
        #
```

```
        # CATEGORY A SKILL: Composition
        # Storing a Game object as a characteristic of the GUI class
        #
        ##############################################################################
        if game is None:
            self._game = Game()
            numPlayers = Game.getNumPlayers()
        else:
            self._game = game[0]
            numPlayers = 2

        # Grid
        WidthInSquares = Game.getWidth() # Dimensions of the grid in squares.
        HeightInSquares = Game.getHeight()
        squareSize = round(self._scale*2.5)

        if network:
            enemyName = self.__getUserInput("EnemyName")
            playerNum, gameID = self._network.send(f"startGame {self._username} {enemyName}").split(" ")
            playerNum = int(playerNum[1]) - 1
            numPlayers = 1
            try:
                myStats = list(map(int, self._network.send(f"getStats {self._username}").split(","))) #
wins,losses,hits,misses,highestHP,leastAttacks
                enemyStats = list(map(int, self._network.send(f"getStats {enemyName}").split(",")))
                needsHelp = self.compareStats(myStats, enemyStats)
                print(myStats, enemyStats)
            except:
                needsHelp = -1
            if needsHelp != -1:
                if needsHelp == 0 and self.__getUserInput(f"Help,Y,N").lower() == "y":
                    self.__options()
                    pg.quit()
                    self.__mainMenu()
                    return
```

```
        elif needsHelp == 1:
            self._network.send(f"deletegame {gameID}")
            playerNum, gameID = self._network.send(f"startGame {self._username} {enemyName}").split(" ")
            playerNum = int(playerNum[1]) - 1


    # Ships
    rects = []
    shipAttach = [0 for _ in range(numships)]
    shipImages = []
    for ship in shipHP:
        shipImages.append(pg.image.load(f"{self._finalImagePath}HP{ship}Ship.png"))
    shipRotation = [0 for _ in range(numships)]
    for image in shipImages:
        rects.append(image.get_rect())

    # Buttons
    buttonImages = [pg.image.load(f"{self._finalImagePath}HP2Ship.png")]
    buttonRects = []
    for x in range(len(buttonImages)):
        buttonRects.append(buttonImages[x].get_rect())
    buttonRects[0].update(0, self._height-(2*buttonRects[0].height), buttonRects[0].width, buttonRects[0].height)


    # Edges of the grid
    left = (self._width - WidthInSquares*squareSize)//2   # Edges are five squares from the centre.
    right = (self._width + WidthInSquares*squareSize)//2
    top = (self._height - HeightInSquares*squareSize)//2
    bot = (self._height + HeightInSquares*squareSize)//2
    squares = []

    # Storing grid object in squares
    for x in range(left, right, squareSize):
        for y in range(top, bot, squareSize):
            squares.append(pg.Rect(x, y, squareSize, squareSize))
```

```
        playerNumberImage = [pg.image.load(f"{self._finalImagePath}P.png"),
pg.image.load(f"{self._finalImagePath}{currentPlayer+1}.png")]

        # Loop for placing ships
        if game is None: # We would skip this part if the game was loaded.
            while runningLocal:
                for event in pg.event.get():
                    if event.type == pg.QUIT:
                        runningLocal = False
                        if network:
                            self._network.send(f"deletegame {gameID}")
                        pg.quit()
                    elif event.type == pg.KEYDOWN:
                        if event.key == pg.K_r:
                            for x in range(len(shipAttach)):
                                if shipAttach[x] == 1:
                                    if shipRotation[x]:
                                        shipRotation[x] = 0
                                    else:
                                        shipRotation[x] = 1
                                    shipImages[x] = pg.transform.rotate(shipImages[x], -90)
                                    rects[x] = shipImages[x].get_rect()
                                    break
                    elif event.type == pg.MOUSEBUTTONDOWN:
                        if event.button == 1 and 1 not in shipAttach:
                            point = pg.mouse.get_pos()
                            changePlayer = True
                            if buttonRects[0].collidepoint(point) and 0 not in shipAttach and 1 not in shipAttach:  #
buttonRects[0] is the done button
                                # place down all ships and swap turns (and move ships back) or move on to actual game.
                                for x in range(len(rects)):
                                    shipLocation = [(rects[x].left - left)//squareSize, (rects[x].top-
top)//squareSize,shipRotation[x], shipHP[x]]
                                    shipResult = self._game.addShip(currentPlayer, shipLocation)
```

```
                        print(shipResult)
                        if shipRotation[x]:
                            shipImages[x] = pg.transform.rotate(shipImages[x], -90)
                            rects[x] = shipImages[x].get_rect()
                            shipRotation[x] = 0
                        rects[x].update(0, 0, rects[x].width, rects[x].height)
                        if shipResult == -1:
                            changePlayer = False

                    if changePlayer:
                        if currentPlayer == 0:
                            if ai != 0:
                                x = 0
                                while x < len(shipHP):
                                    print(x, shipHP)
                                    if self._game.addShip(1,self._game.getAIShip(x)) == 1:
                                        x += 1
                                runningLocal = False
                        if currentPlayer == numPlayers-1:
                            runningLocal = False
                        currentPlayer += 1
                        playerNumberImage[1] = pg.image.load(f"{self._finalImagePath}{currentPlayer+1}.png")
                    else:
                        self._game.clearBoard(currentPlayer)

                for x in range(len(rects)):
                    if rects[x].collidepoint(point): # Checks if the mouse is on one of the ships.
                        # Mouse attaches to ship and starts moving it around.
                        shipAttach[x] = 1
                        break  # Break cause otherwise you can pick up multiple ships at once.
            elif event.type == pg.MOUSEBUTTONUP:
                if event.button == 1 and 1 in shipAttach:
                    point = pg.mouse.get_pos()
                    for x in range(len(shipAttach)):
                        if shipAttach[x] == 1: # If the ship is currently being held, place it down.
```

```
                                    shipAttach[x] = 2

                                    if shipRotation[x]:
                                        rects[x].update(squareSize * round((point[0] - (left % squareSize) -
squareSize/2)/squareSize) + (left % squareSize), squareSize * round((point[1] - (top % squareSize) -
shipHP[x]*squareSize/2)/squareSize) + (top % squareSize), rects[x].width, rects[x].height)
                                    else:
                                        rects[x].update(squareSize * round((point[0] - (left % squareSize) -
shipHP[x]*squareSize/2)/squareSize) + (left % squareSize), squareSize * round((point[1] - (top % squareSize) -
squareSize/2)/squareSize) + (top % squareSize), rects[x].width, rects[x].height)

                                    break

                screen.fill(self._fillColour)
                for x in range(len(shipAttach)):  # Makes a ship follow the mouse around once it has been clicked.
                    if shipAttach[x] == 1:
                        rects[x].update(pg.mouse.get_pos()[0] - rects[x].width/2, pg.mouse.get_pos()[1] - rects[x].height/2,
rects[x].width, rects[x].height)

                for square in squares:
                    pg.draw.rect(screen, (255,255,255), square, 1)

                for x in range (len(rects)):
                    screen.blit(shipImages[x], rects[x])

                for x in range(len(buttonImages)):
                    screen.blit(buttonImages[x], buttonRects[x])

                P, pn = playerNumberImage
                screen.blit(P, (round(screen.get_width()-(6*self._scale)), 0))
                screen.blit(pn, (round(screen.get_width()-(3*self._scale)), 0))
                pg.display.update()

            if network:
                board = self._game.getBoard(0)
```

```
                grid = ""
                for y, row in enumerate(board):
                    for x, value in enumerate(row):
                        if value == 1:
                            grid += f"{x},{y}|"

                grid = grid[:-1]
                self._network.send(f"updateGrids {grid} {gameID} {playerNum}")
                enemyGrid = self._network.send(f"getGrid {gameID} {(playerNum+1)%2}").split("|")
                for coordinate in enemyGrid:
                    x,y = coordinate.split(",")
                    self._game.addShip(1, f"{x}{y}11")

        #
        #
        #
        # ATTACKING STARTS
        #
        #
        #
        currentPlayer = 0
        playerNumbers = [num for num in range(numPlayers)]
        runningLocal = True
        squareSize = min(self._width//(2*WidthInSquares+3), self._height//HeightInSquares) # Scale it down so that two grids
can fit.
        numAttacked = 0
        hits = 0
        misses = 0

        # Edges of the grids
        left = 3*self._scale
        r1 = left + WidthInSquares*squareSize
        right = r1 + squareSize + WidthInSquares*squareSize
        top = int(squareSize)
        bot = int(top + HeightInSquares*squareSize)
```

```
squares = []
enemySquares = []
enemySecretBoard = []
# Storing grid object in squares
for y in range(top, bot, squareSize):
    for x in range(left, r1, squareSize):
        squares.append(pg.Rect(x, y, squareSize, squareSize))

for y in range(top, bot, squareSize):
    for x in range(r1+squareSize, right, squareSize):
        enemySquares.append(pg.Rect(x,y,squareSize,squareSize))

attackTypePaths = [f"{self._finalImagePath}N.png", f"{self._finalImagePath}B.png", f"{self._finalImagePath}R.png"]
attackType = 0
attackTypeImage = pg.image.load(attackTypePaths[attackType])
attackTypeRect = attackTypeImage.get_rect()
attackTypeRect.update(0,0,0,0)
#numbers
numberImages = [pg.image.load(f"{self._finalImagePath}{num}.png") for num in range(10)]

# Game loop
postHit = False
while runningLocal:
    for event in pg.event.get():
        if event.type == pg.QUIT:
            runningLocal = False
            if network:
                self._network.send(f"deletegame {gameID}")
            pg.quit()
        elif event.type == pg.KEYDOWN:
            if postHit:
                postHit = False
                if isinstance(result, list) and len(result) == 1:
                    runningLocal = False
            elif event.key == pg.K_s:
```

```
                    self.__save()
                    runningLocal = False
                elif event.key == pg.K_c:
                    # Change the attacktype
                    attackType = (attackType+1)%3
                    attackTypeImage = pg.image.load(attackTypePaths[attackType])
                    attackTypeRect = attackTypeImage.get_rect()
            elif event.type == pg.MOUSEBUTTONDOWN:
                if event.button == 1:
                    point = pg.mouse.get_pos()
                    if attackTypeRect.collidepoint(point):
                        # Change the attacktype
                        attackType = (attackType+1)%3
                        attackTypeImage = pg.image.load(attackTypePaths[attackType])
                        attackTypeRect = attackTypeImage.get_rect()
                        break
                    for x in range(len(enemySquares)):
                        if enemySquares[x].collidepoint(point): # Checks if the mouse is inside one of the grid squares.
                            # attack that square and swap sides
                            if network:
                                toAttack = 1
                            else:
                                toAttack = playerNumbers[(currentPlayer+1+numAttacked) % numPlayers]

                            coords = [squareSize * round((point[0] - ((left + right//2 )% squareSize) -
squareSize/2)/squareSize) + ((left + right//2) % squareSize), squareSize * round((point[1] - (top % squareSize) -
squareSize/2)/squareSize) + (top % squareSize)]
                            coordinput = [(coords[0] - (left+right//2))//squareSize, (coords[1] - (top))//squareSize]

                            if attackType == 2:
                                result = self._game.radar(toAttack, playerNumbers[currentPlayer])
                                if len(result) == 0:
                                    break
                                postHit = True
                            else:
```

```
                                    result = self._game.attack(coordinput[0], coordinput[1], toAttack, attackType,
playerNumbers[currentPlayer])
                                if result == 2:
                                    numPlayers -= 1
                                    if numPlayers <= 1:
                                        print(f"Player {playerNumbers[currentPlayer]+1} won!")
                                        result = [playerNumbers[currentPlayer]+1]
                                    else:
                                        print(toAttack+1, "was eliminated.")
                                        playerNumbers.remove(toAttack)
                                elif result == 1:
                                    print("Hit!")
                                    hits += 1
                                    pass
                                elif result == -1:
                                    print("Miss!")
                                    misses += 1
                                    pass
                                if result == -2:
                                    # error or already attacked ship
                                    print("Decrementing numAttacked.")
                                    numAttacked -= 1
                                    postHit = False

                                if network and result != -2:
                                    previousAttack = self._network.send(f"updatePreviousAttack {coordinput[0]} {coordinput[1]}
{playerNum} {attackType} {gameID}")
                                    x, y, enemyattackType, pnumignore = map(int, previousAttack.split(" "))
                                    if result == [1]:
                                        postHit = True
                                        break
                                    if enemyattackType != 2:
                                        if self._game.attack(x, y, 0, enemyattackType) == 2:
                                            result = [2]
                                            postHit = True
```

```
                            break # Because we don't want to change the playernumber

                        numAttacked += 1

                        if numAttacked >= numPlayers-1:
                            postHit = True
                            currentPlayer = (currentPlayer + 1) % numPlayers
                            numAttacked = 0
                        if currentPlayer == 1 and ai != 0:
                            airesult = -2
                            print("outside while")
                            while airesult == -2:
                                x,y = self._game.getAIAttack()
                                print("inside while")
                                airesult = self._game.attack(x,y,0,0)

                            if airesult == 2:
                                print("AI won!")
                                result = [2]

                            currentPlayer = (currentPlayer + 1) % numPlayers
                            numAttacked = 0

                        print(playerNumbers,currentPlayer,numAttacked)
                        break # Nothing else to do if a square has been found to be attacked.

            screen.fill(self._fillColour)

            if not postHit:
                if network:
                    myNumber = 0
                    enemyNumber = 1
                else:
                    myNumber = playerNumbers[currentPlayer]
                    enemyNumber = playerNumbers[(currentPlayer+1+numAttacked) % numPlayers]
```

```
                myBoard = self._game.getBoard(myNumber)
                for x in range(len(squares)):
                    row = x // WidthInSquares
                    col = x % WidthInSquares
                    if myBoard[row][col] == 1:
                        pg.draw.rect(screen, (0,255,0), squares[x]) # Green fill for a ship.
                    elif myBoard[row][col] == 0:   # y coord is the % because of how I defined enemySquares.
                        pg.draw.rect(screen, (255,255,255), squares[x], 1) # White grid for an unknown
                    elif myBoard[row][col] == -1:
                        pg.draw.rect(screen, (255,0,0), squares[x]) # Red fill for a hit ship
                    else:
                        pg.draw.rect(screen, (0,0,255), squares[x]) # Blue fill for known empty

                enemySecretBoard = self._game.getSBoard(enemyNumber) # 0 = empty, -1 = hit ship, -2 = hit empty
                for x in range(len(enemySquares)):
                    row = x // WidthInSquares
                    col = x % WidthInSquares
                    if enemySecretBoard[row][col] >= 0:   # y coord is the % because of how I defined enemySquares.
                        pg.draw.rect(screen, (255,255,255), enemySquares[x], 1) # White grid for an unknown
                    elif enemySecretBoard[row][col] == -1:
                        pg.draw.rect(screen, (255,0,0), enemySquares[x]) # Red fill for a hit ship
                    else:
                        pg.draw.rect(screen, (0,0,255), enemySquares[x]) # Blue fill for known empty

                screen.blit(attackTypeImage, attackTypeRect) # Drawing the current attack type, N, B, or R
                screen.blit(numberImages[myNumber+1], (0,6*self._scale)) # Drawing the current player number
                screen.blit(numberImages[enemyNumber+1], (screen.get_width()-3*self._scale, screen.get_height()-
5*self._scale)) # Drawing the other grid's player
            else:
                if isinstance(result, list):
                    if len(result) == 1:
                        screen.blit(numberImages[result[0]], (screen.get_width()//2, screen.get_height()//2))
                    else:
                        x,y = map(str, result)
                        count = 0
```

```
                    for ch in x:
                        ch = int(ch)
                        screen.blit(numberImages[ch], (count*self._scale*3, 0))
                        count += 1

                    count += 1
                    for ch in y:
                        ch = int(ch)
                        screen.blit(numberImages[ch], (count*self._scale*3, 0))
                        count += 1
                pass
            pg.display.update()

        attacks = hits + misses
        if self._username != "":
            if result == [1]:
                self._network.send(f"updateStats {self._username} {1},{0},{hits},{misses},{self._game.getHP(0)},{attacks}")
            else:
                self._network.send(f"updateStats {self._username} {0},{1},{hits},{misses},{0},{10000000000000}")
        if network and result == [2]:
            self._network.send(f"deletegame {gameID}")
        elif game is not None:
            self._network.send(f"deletegame {game[1]}")

    def __addNewResolution(self, newScale):
        ogScale = 2
        scaleIncrease = newScale//ogScale
        print(newScale, scaleIncrease, ogScale)

        if not os.path.isdir(f"FinalImages\\Scale{ogScale*scaleIncrease}"): # Check if the folder already exists.
            # If the folder does not exist, we can create it.
            fileNames = os.listdir(f"FinalImages\\Scale{ogScale}")
            images = []
            for name in fileNames:
                images.append(Image.open(f"FinalImages\\Scale{ogScale}\\{name}"))
```

```
            os.mkdir(f"FinalImages\\Scale{int(ogScale*scaleIncrease)}")
            for x in range(len(images)):
                #resize
                n_size=(images[x].size[0]*scaleIncrease,images[x].size[1]*scaleIncrease) #output size
                res=images[x].resize(n_size,Image.Resampling.NEAREST)

                #save image
                res.save(f"FinalImages\\Scale{ogScale*scaleIncrease}\\{fileNames[x]}") # Scale is relative to 48 * 27 pixels

        self._scale = ogScale * scaleIncrease
        self._width = 48*self._scale
        self._height = 27*self._scale
        self._finalImagePath = f"FinalImages\\Scale{self._scale}\\"
        pg.quit()
        self.__mainMenu()

    def __help(self):
        # Initialise
        pg.init()
        screen = pg.display.set_mode((self._width, self._height))
        runningHelp = True

        # Buttons
        help = pg.image.load(f"{self._finalImagePath}HelpMenu.png")

        while runningHelp:
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    runningHelp = False
                elif event.type == pg.KEYDOWN and event.key == pg.K_ESCAPE:
                    runningHelp = False
            screen.blit(help, (0,0))
            pg.display.update()
```

```
    def __options(self):
        pg.init()
        screen = pg.display.set_mode((self._width, self._height))
        runningOptions = True
        titles = ["resolution", "length", "height", "ShipOne", "ShipTwo", "ShipThree", "ShipFour", "ShipFive", "ShipSix",
"Barrage", "Radar", "Mine", "HitStreak", "numplayers", "colour"]
        changed = {}
        option = pg.image.load(f"{self._finalImagePath}OptionsMenu.png")
        result = ""

        while runningOptions:
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    runningOptions = False
                if event.type == pg.KEYDOWN:
                    if event.key == pg.K_ESCAPE:
                        runningOptions = False
                    elif 48 <= event.key <= 57:
                        result += chr(event.key)
                    elif event.key == pg.K_RETURN:
                        try:
                            toChange = int(result)
                        except:
                            result = ""
                            break
                        result = ""
                        height = Game.getHeight()
                        width = Game.getWidth()
                        try:
                            if toChange != 15:
                                newNum = int(self.__getUserInput(titles[toChange-1]))
                        except Exception as e:
                            print(e, toChange)
                            break
                        print(toChange, newNum)
```

```
                    if toChange == 1:
                        self.__addNewResolution(newNum)
                        runningOptions = False
                    elif toChange == 14:
                        if newNum > 1:
                            Game.changeNumPlayers(newNum)
                        else:
                            print("Must be 2 or more players")
                        changed[int(toChange)-1] = newNum # No hints in GUI mode D: IMPORTANT PROBLEM FIX.
                    else:
                        changed[int(toChange)-2] = newNum

                    if toChange == 2:
                        Game.changeBoardDimensions(newNum, height)
                    elif toChange == 3:
                        Game.changeBoardDimensions(width, newNum)
                    elif 4 <= toChange <= 9:
                        Game.changeShips(toChange - 4, newNum)
                    elif 10 <= toChange <= 13:
                        Game.changeSpecials(toChange-10, newNum)
                    elif toChange == 15:
                        try:
                            red = int(self.__getUserInput("Red"))
                            blue = int(self.__getUserInput("Blue"))
                            green = int(self.__getUserInput("Green"))
                        except ValueError:
                            break
                        self._fillColour = (red, blue, green)

            screen.blit(option, (0,0))
            pg.display.update()

        self.updateOptions(changed)
```

```
    def __load(self):
        if self._username is not None:
            loads = self._network.send(f"getLoads {self._username}")
        else:
            return

        choice = "a"
        while not choice.isnumeric():
            print("".join(loads), loads)
            choice = self.__getUserInput("".join(loads))
            if choice == "":
                return
        grids = self._network.send(f"loadGame {choice}")
        grid1, grid2, ai = grids.split(" ")
        ai = int(ai)
        Game.changeAI(ai)
        game = Game()

        for coordinate in grid1.split("|"):
            x,y = coordinate.split(",")
            game.addShip(0, f"{x}{y}{0}{1}")

        for coordinate in grid2.split("|"):
            x,y = coordinate.split(",")
            game.addShip(1, f"{x}{y}{0}{1}")

        self.__local(False, ai, [game, False])

    def __save(self):
        if self._username is not None:
            print("Must be logged in")
            return
        board1 = self._game.getBoard(0)
        grid1 = ""
        for y, row in enumerate(board1):
```

```
            for x, value in enumerate(row):
                if value == 1:
                    grid1 += f"{x},{y}|"
        grid1 = grid1[:-1]

        board2 = self._game.getBoard(1)
        grid2 = ""
        for y, row in enumerate(board2):
            for x, value in enumerate(row):
                if value == 1:
                    grid2 += f"{x},{y}|"
        grid2 = grid2[:-1]

        self._network.send(f'addGame {grid1} {grid2} {self._username} Guest ""')

    def __authenticate(self, authtype):
        pg.init()
        screen = pg.display.set_mode((self._width, self._height))
        runningAuthenticate = True
        # Buttons
        rects = [None, None]
        rects[0] = pg.Rect(16*self._scale, 11*self._scale, 19*self._scale, 6*self._scale)
        rects[1] = pg.Rect(17*self._scale, 19*self._scale, 17*self._scale, 6*self._scale)
        if authtype == "login":
            backGround = pg.image.load(f"{self._finalImagePath}LoginMenu.png")
        else:
            backGround = pg.image.load(f"{self._finalImagePath}SignUpMenu.png")

        password = ""
        while runningAuthenticate:
            for event in pg.event.get():
                if event.type == pg.QUIT:
                    runningAuthenticate = False
                elif event.type == pg.KEYDOWN and event.key == pg.K_ESCAPE:
                    runningAuthenticate = False
```

```
                elif event.type == pg.MOUSEBUTTONDOWN and event.button == 1:
                    mouseLocation = pg.mouse.get_pos()
                    for index, rect in enumerate(rects):
                        if rect.collidepoint(mouseLocation):
                            if index == 0:
                                self._username = self.__getUserInput("USERNAME")
                            else:
                                password = self.__getUserInput("PASSWORD")

                            if self._username != "" and password != "":
                                if authtype == "login":
                                    valid = self._network.send(f"login {self._username} {self.hash(password +
self._username)}")
                                    if valid == "false":
                                        self._username =  ""
                                        password = ""
                                    else:
                                        runningAuthenticate = False
                                else:
                                    valid = self._network.send(f"signUp {self._username} {self.hash(password +
self._username)}")
                                    if valid == "not_unique":
                                        self._username = ""
                                        password = ""
                                    else:
                                        runningAuthenticate = False

            screen.blit(backGround, (0,0))
            pg.display.update()

        if self._username != "":
            self.__playMenu()
        else:
            return
```

```
def __getUserInput(self, title): # returns a string
    runningUserInput = True
    re = ""
    screen = pg.display.set_mode((self._width, self._height))

    underline = pg.image.load(f"{self._finalImagePath}Underline.png")
    titleLetters = []
    for letter in title:
        titleLetters.append(pg.image.load(f"{self._finalImagePath}{letter.upper()}.png"))

    letterImages = []
    delta = 0
    while runningUserInput:
        for event in pg.event.get():
            if event.type == pg.QUIT:
                runningUserInput = False
            elif event.type == pg.KEYDOWN:
                if event.key == pg.K_BACKSPACE:
                    re = re[:-1]
                    letterImages.pop()
                elif event.key == pg.K_LSHIFT:
                    if delta == 0:
                        delta = -32
                    else:
                        delta = 0
                elif event.key == pg.K_ESCAPE or event.key == pg.K_RETURN:
                    runningUserInput = False
                elif event.key == pg.K_CAPSLOCK:
                    if delta == 0:
                        delta = -32
                    else:
                        delta = 0
                else:
                    try:
                        chr(event.key).isalpha()
```

```
                    re += chr(event.key+delta)
                    print(event.key+delta, delta, re)
                    letterImages.append(pg.image.load(f"{self._finalImagePath}{chr(event.key).upper()}.png"))
                except ValueError:
                    pass
            elif event.type == pg.KEYUP:
                if event.key == pg.K_LSHIFT:
                    if delta == 0:
                        delta = -32
                    else:
                        delta = 0

        screen.fill(self._fillColour)
        position = 0
        for index, letter in enumerate(titleLetters):
            if title[index].upper() == "M" or title[index].upper() == "W":
                letterSize = 6
            elif title[index].upper() == "Q":
                letterSize = 5
            elif title[index].upper() == "I":
                letterSize = 2
            elif title[index].upper() == ",":
                letterSize = 1
            else:
                letterSize = 4
            if title[index].isupper():
                screen.blit(underline, (position, round(7.5*self._scale)))
            screen.blit(letter, (position, self._scale))
            position += self._scale*letterSize

        position = 0
        for index, letter in enumerate(letterImages):
            if re[index].upper() == "M" or re[index].upper() == "W":
                letterSize = 6
            elif re[index].upper() == "Q":
```

```
                letterSize = 5
            elif re[index].upper() == "I":
                letterSize = 2
            else:
                letterSize = 4
            if re[index].isupper():
                screen.blit(underline, (position, round(self._height//2 + 6*self._scale)))
            screen.blit(letter, (position, self._height//2))
            position += self._scale*letterSize
        pg.display.update()

    print("DEBUG -- Returning re", re)
    return re

################################################################################
#
# CATEGORY A SKILL: Inheritance
# The Terminal inherits from the base UI class
#
################################################################################
class Terminal(UI):
    def __init__(self):
        super().__init__()

    def run(self):
        self.__mainMenu()

    def __mainMenu(self):
        while True:
            print(text2art("Battleships"))
            print("Main Menu:")
            print("1. Log in")
            print("2. Sign up")
            print("3. Guest")
            print("4. Statistics")
```

```
            print("5. Quit")

            opt = input()
            if opt == "1":
                self.__logIn()
            elif opt == "2":
                self.__signUp()
            elif opt == "3":
                self.__playMenu()
            elif opt == "4":
                self.__stats()
            else:
                break

    def __signUp(self):
        valid = "not_unique"
        while valid == "not_unique":
            print("--The username must be unique--")
            self._username = input("Username - ")
            password = input("Password - ")
            if " " in self._username or " " in password:
                print("Username or password cannot have spaces!")
            else:
                valid = self._network.send(f"signUp {self._username} {self.hash(password + self._username)}")
            print("")

        self.__playMenu()

    def __logIn(self):
        valid = "false"
        while valid == "false":
            self._username = input("Username - ")
            password = input("Password - ")
            print("")
            valid = self._network.send(f"login {self._username} {self.hash(password + self._username)}")
```

```
        self.__playMenu()

    def __stats(self):
        if self._username != "":
            stats = self._network.send(f"getStats {self._username}")
        print("Format: wins,losses,hits,misses,highestHP,leastAttacks")
        input(stats)

    def __playMenu(self):
        while True:
            print(text2art("Battleships"))
            print("1. Local")
            print("2. Network")
            print("3. AI Opponent")
            print("4. Help")
            print("5. Options")
            print("6. Load Game")
            print("7. Quit")
            opt = input()
            if opt == "1":
                self.__local(0) # No ai
            elif opt == "2":
                self.__networkGame()
            elif opt == "3":
                self.__local(input("What AI difficulty? 1,2 or 3"))
            elif opt == "4":
                self.__help()
            elif opt == "5":
                self.__options()
            elif opt == "6":
                self.__load()
            else:
                break
```

```
    def __getShipInput(self, playerNumber, ai, y): # x = player number, ai = ai difficulty, y = index of ship to be placed
        if int(ai) != 0:
            if playerNumber == 1:
                return self._game.getAIShip(y)
        return input((f"Ship in format x,y,r")).split(",")

    def __getAttackInput(self, playerNumber, ai): # x = player number, ai = ai difficulty
        if int(ai) != 0:
            if playerNumber == 1:
                return [self._game.getAIAttack(), 0]

        attackType = "a"
        specialCounts = self._game.getSpecialCount(playerNumber)[:2]
        while not attackType.isnumeric() or (attackType != "0" and specialCounts[int(attackType)-1] == 0):
            print("Your specials: [barrageCount, radarCount]")
            print(specialCounts) # Don't care about mines or hitstreak
            print("0 - normal, 1 - barrage, 2 - radar")
            attackType = input("What attack do you want to use?")
            print("")

        attackType = int(attackType)

        if attackType <= 1:
            coordinates = ["a"]
            while len(coordinates) < 2 or not coordinates[0].isnumeric() or not coordinates[1].isnumeric():
                coordinates = input("Input square to attack (x,y), can't attack the same square twice.").split(",")
            return coordinates, attackType
        return 0, attackType

    def __local(self,aiDifficulty, game=None): # aiDifficulty is 0,1,2 or 3, 0 for no ai, and then 1,2,3 for difficulty level.
        ##############################################################################
        #
        # CATEGORY A SKILL: Composition
        # Storing a Game object as a characteristic of the Terminal class
        #
```

```
        ##########################################################################
        if game is None:
            Game.changeAI(aiDifficulty)
            print("AI difficulty:", aiDifficulty)
            self._game = Game()
            numPlayers = Game.getNumPlayers()
        else:
            self._game = game[0]
            numPlayers = 2

        hints = []
        if Game.getSpecials()[4]:
            hints = [
                    ["Don't place ships to close to eachother.", "Be unpredictable", "Don't repeat what you have done
previously.", "Place ships further from centre."],
                    ["The centre is most likely to contain ships", "Attack around an already hit ship to finnish it off.",
"Attack in diagonals."]
                    ]

        if self._username != "":
            attacks = 0
            hits = 0
            misses = 0

        if game is None:
            for playerNum in range(numPlayers): # Placing ships.
                print(f"Player {playerNum+1} turn")
                pp(self._game.getBoard(playerNum))

                numShips = Game.getNumShips()
                ships = Game.getShips()

                for ship in ships:
                    if len(hints):
                        print(random.choice(hints[0]))
```

```
                print(f"Player {playerNum+1} to place down ship of length {ship}")

                shipLocation = self.__getShipInput(playerNum, aiDifficulty, numShips-1)
                shipLocation.append(ship)
                while self._game.addShip(playerNum, shipLocation) == -1:
                    print("Invalid ship position, current state of board:")
                    pp(self._game.getBoard(playerNum))
                    shipLocation = self.__getShipInput(playerNum, 0, numShips-1)
                    shipLocation.append(ship)

                numShips -= 1

            pp(self._game.getBoard(playerNum))

            input("Press enter to change turns and clear terminal")
            os.system('cls')

        print("Time to attack!")

    print(numPlayers)
    playerNumbers = [num for num in range(numPlayers)]
    play = True
    while play:
        choice = input("Write 's' to save, anything else to continue")
        if choice.lower() == "s":
            if numPlayers != 2:
                print("Can only save two player games.") # IMPORANT FIX PROBLEM
            self.__save()
            break
        for playerNum in range(len(playerNumbers)):
            if not play:
                break
            print(f"Player {playerNumbers[playerNum]+1} turn")
            print(f"Player {playerNumbers[playerNum]+1} board:")
            pp(self._game.getBoard(playerNumbers[playerNum]))
```

```
                if len(hints):
                    print(random.choice(hints[1]))
                for deltaPlayer in range(1, numPlayers):
                    toAttack = playerNumbers[(playerNumbers[playerNum]+deltaPlayer) % numPlayers]

                    print(f"Player {(toAttack) + 1} secret board:")
                    for row in self._game.getSBoard(toAttack): # it returns a 2d array, so we print the rows.
                        print(row)

                    result = -2
                    while result == -2: # Returing -2 = already hit square or error, -1 = miss, 1 = hit ship, 2 = game won
                        attackLocation, attackType = self.__getAttackInput(playerNumbers[playerNum], aiDifficulty)
                        result = -2
                        if attackType == 2:
                            result = self._game.radar(toAttack, playerNumbers[playerNum])
                            print(f"There is a ship at x={result[0]}, y={result[1]}")
                        else:
                            try:
                                result = self._game.attack(int(attackLocation[0]), int(attackLocation[1]), toAttack,
attackType, playerNumbers[playerNum]) # Index starts at 0
                            except IndexError:
                                print("Invalid attack format")

                    if result == 2:
                        numPlayers -= 1
                        if numPlayers == 1:
                            print("Hit!")
                            print(f"Player {playerNumbers[playerNum]+1} wins!")
                            play = False
                            break
                        else:
                            print(toAttack+1, "was eliminated.")
                            playerNumbers.remove(toAttack)
                    elif result == 1:
```

```
                    print("Hit!")
                elif result == -1:
                    print("Miss!")

                if playerNumbers[playerNum] == 0 and aiDifficulty and self._username:
                    attacks += 1
                    if result == 2:
                        hits += 1
                        self._network.send(f"updateStats {self._username}
{1},{0},{hits},{misses},{self._game.getHP(0)},{attacks}")
                    elif result == 1:
                        hits += 1
                    else:
                        misses += 1
                elif result == 2 and aiDifficulty and self._username: # AI won
                    self._network.send(f"updateStats {self._username}
{0},{1},{hits},{misses},{self._game.getHP(0)},{attacks}")

            input("Press enter to change turns and clear terminal")
            os.system('cls')

        if game is not None:
            self._network.send(f"deletegame {game[1]}")

    def __networkGame(self):
        ##############################################################################
        #
        # CATEGORY A SKILL: Composition
        # Storing a Game object as a characteristic of the Terminal class
        #
        ##############################################################################
        self._game = Game()
        if self._username == "":
            print("Must be logged in!")
            return
```

```
        print(text2art("Online"))
        enemyName = input("What is your enemy's username?")
        print("Setting up game with opponent...")
        playerNum, gameID = self._network.send(f"startGame {self._username} {enemyName}").split(" ")
        playerNum = int(playerNum[1]) - 1

        # get and compare player statistics
        hits = 0
        misses = 0

        try:
            myStats = list(map(int, self._network.send(f"getStats {self._username}").split(","))) #
wins,losses,hits,misses,highestHP,leastAttacks
            enemyStats = list(map(int, self._network.send(f"getStats {enemyName}").split(",")))
            needsHelp = self.compareStats(myStats, enemyStats)
            print(myStats, enemyStats)
        except:
            needsHelp = -1

        if needsHelp != -1:
            print(f"It is recommended that player {needsHelp+1} gets help.")
            if needsHelp == playerNum:
                if input("Do you want to change your options to make the game easier? (Y/N)").upper() == "Y":
                    self.__options()
                else:
                    print("OK, continue as is. Good luck!")
            else:
                print("Checking if enemy player wants to get help.")

        pp(self._game.getBoard(playerNum))
        numShips = Game.getNumShips()
        ships = Game.getShips()

        for ship in ships:
            print(f"Place down a ship of length {ship}")
```

```
            shipLocation = self.__getShipInput(playerNum, 0, numShips-1)
            shipLocation.append(ship)
            while self._game.addShip(playerNum, shipLocation) == -1:
                print("Invalid ship position, current state of board:")
                pp(self._game.getBoard(playerNum))
                shipLocation = self.__getShipInput(playerNum, 0, numShips-1)
                shipLocation.append(ship)
            numShips -= 1

        board = self._game.getBoard(playerNum)
        grid = ""
        for y, row in enumerate(board):
            for x, value in enumerate(row):
                if value == 1:
                    grid += f"{x},{y}|"

        grid = grid[:-1]
        self._network.send(f"updateGrids {grid} {gameID} {playerNum}")
        enemyGrid = self._network.send(f"getGrid {gameID} {(playerNum+1)%2}").split("|")
        for coordinate in enemyGrid:
            x,y = coordinate.split(",")
            self._game.addShip((playerNum+1) % 2, f"{x}{y}11")
        # Ships have been placed
        # Start attacking:
        play = True
        while play:
            print("Your board:")
            pp(self._game.getBoard(playerNum))
            print("")
            print("Enemy secret board")
            pp(self._game.getSBoard((playerNum+1)%2))
            result = -2
            while result == -2: # Returing -2 = already hit square or error, -1 = miss, 1 = hit ship, 2 = game won
                attackLocation, attackType = self.__getAttackInput(playerNum, 0)
                result = -2
```

```
                if attackType == 2:
                    result = self._game.radar((playerNum+1) % 2)
                    print(f"There is a ship at x={result[0]}, y={result[1]}")
                else:
                    try:
                        result = self._game.attack(int(attackLocation[0]), int(attackLocation[1]), (playerNum+1) % 2,
attackType) # Index starts at 0
                    except IndexError:
                        print("Invalid attack format")
            if result == 2:
                hits += 1
                print("Hit!")
                print(f"Player {playerNum+1} wins!")
                play = False
                self._network.send(f"updateStats {1},{0},{hits},{misses},{0},{10000000}")
            elif result == 1:
                hits += 1
                print("Hit!")
            elif result == -1:
                misses += 1
                print("Miss!")

            previousAttack = self._network.send(f"updatePreviousAttack {attackLocation[0]} {attackLocation[1]} {playerNum}
{attackType} {gameID}")
            print("Enemy player attacked - ", previousAttack)
            x, y, attackType, pnumignore = map(int, previousAttack.split(" "))
            result = self._game.attack(x, y, playerNum, attackType)
            if result == 2:
                self._network.send(f"updateStats {0},{1},{hits},{misses},{0},{10000000}")
                play = False

        self._network.send(f"deletegame, {gameID}")

    def __help(self):
        print("\n")
```

```
        print(text2art("RULES"))
        print("The game consists of consists of two phases, the preparation and attack phases.")
        print("In the preperation phase, players place own all of their ships on a grid (often 10x10).")
        print("-------------------------------------------------------------------------------------")
        print("In the attack phase, it is time to attack the other players' ships")
        print("Do this by choosing a square on the grid that you think contains an enemy ship")
        print("You will be told whether your attack was a hit, then it changes to another players turn")
        print("-------------------------------------------------------------------------------------")
        print("This cycle continues until one player has destroyed all of the enemies ships.")
        print("The last player with ships remaining wins!")
        input("Press any button to continue.")

    def __options(self):
        changed = {}
        while True:
            os.system('cls')
            ships = Game.getShips()
            width = Game.getWidth()
            height = Game.getHeight()
            numShips = [0,0,0,0,0,0]
            for ship in ships:
                numShips[ship-1] += 1
            specialsCounts = Game.getSpecials()
            numPlayers = Game.getNumPlayers()

            print("-----Options Menu-----")
            print(f"1. Grid Width {width}")
            print(f"2. Grid Height {height}")
            print(f"3. Ship One {numShips[0]}")
            print(f"4. Ship Two {numShips[1]}")
            print(f"5. Ship Three {numShips[2]}")
            print(f"6. Ship Four {numShips[3]}")
            print(f"7. Ship Five {numShips[4]}")
            print(f"8. Ship Six {numShips[5]}")
```

```
print("-----Special Mode-----")
print(f"9. Barrage {specialsCounts[0]}")
print(f"10. Radar {specialsCounts[1]}")
print(f"11. Mines {specialsCounts[2]}")
print(f"12. Hit Streak {specialsCounts[3]}")
print(f"13. Hints {specialsCounts[4]}")
print(f"14. Number of players {numPlayers}")
print("")
print("Write the number of the thing you want to change, followed by what it should be changed to. (e.g. 11 -1)")
print("Write 'E' to exit")
inp = input()

toChange = inp.split(" ")[0]
if toChange == "E":
    break

if len(inp.split(" ")) == 2:
    newNum = inp.split(" ")[1]
    try:
        newNum = int(newNum)
        toChange = int(toChange)
    except:
        input("Invalid type")
        continue

#############################################################################
#
# CATEGORY B SKILL: Dictionaries
# Adding a key value pair to a dictionary. This stores the option to be
# changed, and what it should be changed to.
#
#############################################################################
if toChange < 15:
    changed[toChange-1] = newNum
else:
```

```
                continue

            if toChange == 1:
                Game.changeBoardDimensions(newNum, height)
            elif toChange == 2:
                Game.changeBoardDimensions(width, newNum)
            elif 3 <= toChange <= 8:
                Game.changeShips(int(toChange) - 3, newNum)
            elif 9 <= toChange <= 13:
                Game.changeSpecials(toChange-9, newNum)
            elif toChange == 14:
                if newNum > 1:
                    Game.changeNumPlayers(newNum)
                else:
                    print("Must be 2 or more players")
            Game.updatePlayers()

        self.updateOptions(changed)

    def __load(self):
        if self._username is not None:
            loads = self._network.send(f"getLoads {self._username}")
        for load in loads.split(","):
            print(load)

        choice = input("Which game would you like to load?")
        if choice.upper() == "E":
            return
        grids = self._network.send(f"loadGame {choice}")
        grid1, grid2, ai = grids.split(" ")

        Game.changeAI(ai)
        game = Game()

        for coordinate in grid1.split("|"):
```

```
            x,y = coordinate.split(",")
            game.addShip(0, f"{x}{y}{0}{1}")

        for coordinate in grid2.split("|"):
            x,y = coordinate.split(",")
            game.addShip(1, f"{x}{y}{0}{1}")

        self.__local(ai, [game, int(choice)])

    def __save(self):
        if self._username is None:
            print("Must be logged in")
            return
        board1 = self._game.getBoard(0)
        grid1 = ""
        for y, row in enumerate(board1):
            for x, value in enumerate(row):
                if value == 1:
                    grid1 += f"{x},{y}|"
        grid1 = grid1[:-1]

        board2 = self._game.getBoard(1)
        grid2 = ""
        for y, row in enumerate(board2):
            for x, value in enumerate(row):
                if value == 1:
                    grid2 += f"{x},{y}|"
        grid2 = grid2[:-1]

        self._network.send(f'addGame {grid1} {grid2} {self._username} Guest ""')
```

This should be fully commented and broken into suitable sections with subheadings. You can turn this section to landscape to make it easier to read. Ensure this is **fully annotated** with comments. Can use a website like hilite.me. Do not use 'dark mode' images or screenshot

4 System testing *(8 marks)*

4.1 Test plan

Objective(s) Tested	Test Data	Expected Outcome	Result and timestamp
1	“python scripts\Battleships.py t”	The terminal version runs and prints a menu	
2	“9,7,1”, “0,0,0”, “1,1,1”, “5,5,0”	All of the ships should be placed down and displayed to the user.	
4,5	“5,5”	The boards should be displayed to the user and the enemy should be attacked at 5,5	
6	N/A	Turns should change appropriately	
7	N/A	Once a player has won, they should be declared the winner.	
8	“python scripts\Battleships.py a”	Should display an error and tell the user how to properly start the game	
9	“200,1000,20”, “4,4,3”, “1231231”, “a,b,c”, “5,5,0”, “5,5,0”	Only the first “5,5,0” should be accepted.	
10,11	“5,5” on a previous turn “100,100”, “a,c”, “5,5”, “2,2”	Only the “2,2” should be accepted. An “invalid attack” error should be displayed, and the user should try again.	
12	N/A	After a user ends their turn the terminal should be wiped.	
13	N/A	IMPORTANT PROBLEM FIX	
14,15	“1 15”, “2 6”, “5 4”, “4 0”, “9 6”, “14 5”, “929014 abc”, “abc iu”, “123 4”	The grid size, ships, barrages and number of ships should change. The last three inputs should not be accepted.	
16	“python scripts\Battleships.py g”	The main menu should be displayed.	

17,18,19,20	Mouse movement and clicks followed by pressing the “Esc” key	Each of the menus should be displayed, then the game should return to the main menu	
21	Clicking on the local option.	A game should start and the ship, done button and grids should be displayed.	
22,23,24,25	Dragging and dropping ships into invalid positions and pressing the done button. Followed by dragging and dropping and rotating ships into valid options and pressing the done button.	On the first done button press, the game should reject it and make the user repeat the ship placing. The second done button press should be accepted and turns should change.	
27,28,29,30	Clicking the ready button then clicking somewhere on the right grid to attack.	Nothing should happen unless the user clicks on the grid. Upon clicking on a valid square, that square should be attacked and turns should change	
31	Attacking on the grid	If the attack hit, that square should turn red, otherwise it will turn blue.	
32,33,34	Repeat attacks until the game ends.	The winner should be displayed, and the game should return to the play menu.	
35	Changing the resolution in the options menu.	The resolution should change, and a different looking window will appear.	
36	Test whether the user can navigate all menus correctly.	All menus work as intended and all features are accessible and clear.	
37	Press any button to see if the blank screen goes away after a turn.	The blank screen goes away and turns change.	
38	Press the help button in the play menu.	Information about the game should be displayed.	

39,40,41,42,43,44,45, 46,47	Enter the options menu and start changing all the settings.	All options should be changeable.	
48	In a game, select the barrage attack and attack a square	A 3x3 grid of square should be attacked.	
49	N/A	N/A	
50	In a game, select the radar attack and click anywhere on the enemy grid.	The location of an enemy ship should be displayed.	
51	Select AI in the play menu.	The user should be prompted for the ai difficulty.	
52	Enter 1,2 or 3 in the ai difficulty screen.	A game against that level of AI should start.	
53	Play against level 3 AI.	When attacking starts, the AI should start attacking at the centre	
54	Play against level 1 or 2.	When attacking starts, the AI should start attacking a bit randomly, but near the centre.	
55	Wait until the AI hits a ship	On the next turns, the AI should start attacking orthogonally around the hit ship.	
56	Wait until the AI has destroyed one side of a ship.	On the next turns, the AI should start destroying the other side of the ship.	
57	Go to the options menu and change the number of players.	The options menu should accept any number greater than 1. When starting a game after this, the number of players should be remembered.	
58,59,60,61	Play a multiple person game.	Each player attacks all other players, and a player with no remaining ships is eliminated and that the final player is declared the winner.	

62	Press “S” in the middle of an attack phase when you are connected to the server.	The game should be saved, and the menu returns to the play menu.	
62	Press “S” when you are not connected to the server	The game continues and is not saved.	
64,65	Select the load option from the main menu.	The game IDs of saved games should be displayed.	
66	N/A	N/A	
68	Press Escape in the load game menu	This should return to the play menu	
69,70	Open the main menu	An option to login, signup or play as guest should exist in the main menu.	
71,73,74	Input a as the name and b as the password	The game should go to the play menu, and the user should be able to do everything which requires logging in.	
75,76,77	Input a as the name and b as the password Then input a unique username and password combination.	“a” and “b” should be rejected, and the second versions should be accepted.	
80,83,84	Open the statistics menu and then press “Esc”.	The statistics should be displayed, then the user should return to the main menu.	
85,86,87	Play a 100% win rate account against a 0% win rate account.	It is suggested that the 0% win rate player gets additional help.	
88,89	Connect two devices to each other through a server	An online game starts	
90	Start placing ships on both devices	The game does not start until both players have pressed the done button	
91,2	Attack on both devices.	Both attacks should go through, but the game waits if one player is	

		one attack ahead of the other.	
93	Change options.	Options carry over to the network game (e.g. special attacks, ships, etc)	

4.2 Testing Videos

I made it but have not been able to upload it yet.

Record your program being used, narrate each objective being shown to have been implemented. Give the objective in the audio of the recording. Provide bit.ly shortened links in the document to your videos which have been uploaded to youtube. Use a large monospace (e.g courier new) font with no underlining e.g.

https://bit.ly/abc0DEF12

5 Evaluation *(4 marks)*

5.1 Comparison of project performance against the objectives

Overall, there were five objectives which were partially met and two objectives which were not met at all. The rest were entirely met.
In Stages 0,1,2, 3, 5, and 6 all objectives were met.

Stage 4 Full Game:
This stage contains both objectives which I could not meet and one objective which I was only able to partially meet. The objectives I could not meet are about adding a mine and hit streak to the game. If I had more time, I believe I could be able to implement these, however I did not think that these were vital for the game and decided to focus on other features.

Stage 7 AI Opponent:
There was one objective which I only partially met. Objective 66 which states that when a player loads a game, they should be sent to play against the correct opponent. I was able to allow game saves, however I could not guarantee that the game was started against the correct online opponent or AI. When loading a game, it only loads the game as a local game. I do not believe that this is a large problem as it is not a specific thing my client requested, however I believe that improving the saves in this manner would help the user and increase the quality of the game.

Stage 8 User accounts:
In this stage there was one objective which I could only partially meet. Objective 86 states that before an online game starts, the game should check if either player needs help by comparing their previous games against each other, total win rate and hit rate. While I was able to implement this idea, the game only compares their overall win rate and hit rate, but not previous games against each other. I decided to not implement this part of the objective because it would increase the amount of things stored in the database by a lot and I did not believe that it was an important enough objective for that.

Stage 9 Network Games:
I had initially planned on not adding network games as they seemed quite complicated and online games were not required by my client. However, I am very glad that I was able to implement them. While I was able implement online games, I did not have time to make everything which works in local games work in online games, and therefore could only partially complete objective 93: “Everything which can be done in a local game should be possible in an online game”. I was not able to implement >2 player games. If I had more time this probably could have been down, however generalising it for n players would have been quite difficult. In addition, the client who had requested 4 player games did not care about online games, so I do not believe that this is a major problem.

5.2 Effectiveness of the solution

I am happy that I was able to meet all of the requirements of a Battleships game and all of the basic requirements of my users. However, there were some more advanced features which clients had suggested that I was not always able to meet in the most optimal way. For example, my clients had requested a special game mode, radar and barrage attack. I was able to implement all of this, but I think that my clients would have preferred if I had gone further and added more special ships/attacks/defences etc.

A similar problem can be seen in my GUI. It is fully functional, and all features can be seen and accessed, however it is not the most intuitive or visually pleasing GUI. For example, when typing, there are certain characters which may not be used, and this is not stated anywhere and could lead to confusion when the characters do not appear on the screen. In addition I am not certain that the logos for the options, help and statistics menu are clear enough, however upon clicking on them it should be clear what they are. Finally, the settings in the options menu can only be modified using the keyboard, and the current settings are not displayed to the user.

For the online games and user accounts there is a problem of having to have a server always running. Currently one of the users would have to act as the server by running the server.py file. While this works,

I believe that I could have made it easier for the user to act as a host through the main game, instead of making the user have to run an additional python file.

Finally, I have realised that using vector graphics may have been a better solution to allow the user to change the resolution of the game. This could have reduced the number of files created as currently each image is copied over to the new resolution, and creating a new resolution requires the creation of 60 new image files. For 1920*1080 the folder containing these files is 37 kilobytes and takes less than a second to create.

5.3 Analysis of user feedback

In order to receive user feedback, I gave my client copies of the required field and the user guide contained in this document. After they had played the game, I asked what could be improved about it. They found a few bugs which I had not caught, and I have now fixed those problems. The feedback could be categorised into three parts, GUI, the game itself, and AI. For each of these I asked for a score out of 10.

GUI:

Score	New Features	Overall
6/10	• Fullscreen • Animations • Sound • New characters when typing • More detailed images	They were a little bit confused navigating the menus and its not clear where you can use your mouse or where you have to type. To improve the quality, adding more detailed images, animations and sound could go a long way. It was not clear which characters could be used, and would prefer if special characters could be used as those are often used for passwords.

AI:

Score	New Features	Overall
8/10	• Allow the AI to play in >2 player games. • Allow the AI to use special attacks.	The AI was good and challenging. Would be more fun if they could play with their friends against the AI, or the AI against each other. In addition, the AI should know how to use special attacks.

Game:

Score	New Features	Overall
10/10		They were very happy with the amount of features within the game, and that all the requested

		features and more were included.

This should include the original user who was involved at the analysis stage

5.4 Possible improvements

My client and I believe that the base game is quite good. The largest improvements I would suggest are in the GUI. The most important part is to make the GUI look and feel better. First, I would consider adding more detailed and better drawn images. After that I think that animations, sound effects and music would go a far way to make the game feel more polished. This is, of course, easier said than done. To get more detailed images I would consider vector graphics and seeing if there are any good free to use images which can be found. I don't think that the animations need to be very complicated. The most basic thing to add would be highlighting the squares/menu options which the player is hovering over. After that, an animation showing the attacks going through could be added.
For music, I think there should be calm menu music, and exciting gameplay music. In addition, a fairly simple addition of a Fullscreen option would be very useful.
If these improvements were made, I believe that the GUI would be complete.

The only improvements my user had for the game were in regards to the AI. In particular allowing the AI to play against itself and in special games.
I agree with my client, however I believe that the online games could also be improved by allowing for >2 player games. In addition I believe that improvements can be made to the base game by adding more types of ships, new attacks and defences and different grid sizes.
Be verbose about what could be improved – make a full statement. Get user feedback for improvement

6 Appendix

6.1 Git log

6.2 References to web sites or other resources used

Group	Model (including data model/structure)	Algorithms
A	Complex data model in database (eg several interlinked tables)	Cross-table parameterised SQL
		Aggregate SQL functions
		User/CASE-generated DDL script
	Hash tables, lists, stacks, queues, graphs, trees or structures of equivalent standard	Graph/Tree Traversal
		List operations
	Files(s) organised for direct access	Linked list maintenance
		Stack/Queue Operations
		Hashing
	Complex scientific/mathematical/robotics/ control/business model	Advanced matrix operations
		Recursive algorithms
		Complex user-defined algorithms (eg optimisation, minimisation, scheduling, pattern matching) or equivalent difficulty
		Mergesort or similarly efficient sort
	Complex user-defined use of object-orientated programming (OOP) model, eg classes, inheritance, composition, polymorphism, interfaces	Dynamic generation of objects based on complex user-defined use of OOP model
		Server-side scripting using request and response objects and server-side extensions for a complex client-server model
	Complex client-server model	Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model

Group	Model (including data model/structure)	Algorithms
B	Simple data model in database (eg two or three interlinked tables)	Single table or non-parameterised SQL
	Multi-dimensional arrays Dictionaries Records	Bubble sort Binary search
	Text files File(s) organised for sequential access	Writing and reading from files
	Simple scientific/mathematical /robotics/ control/business model	Simple user defined algorithms (eg a range of mathematical/statistical calculations)
		Generation of objects based on simple OOP model
	Simple OOP model Simple client-server model	Server-side scripting using request and response objects and server-side extensions for a simple client-server model
		Calling Web service APIs and parsing JSON/ XML to service a simple client-server model
C	Single-dimensional arrays	Linear search
	Appropriate choice of simple data types Single table database	Simple mathematical calculations (eg average)
		Non-SQL table access